APPLICATIONS OF GENERATIVE MODELS
TO ENHANCE THE PERFORMANCE OF COMPLEX PROBABILISTIC SEISMIC MODELS

By

Thong Minh Nguyen (Martin) – 68623131

Jian Di – 61972461

Years 2019 – 2021

Report submitted in part fulfilment of the requirement for the

Applied Data Science Project

Master of Applied Data Science

In the School of Mathematics and Statistics

at

UNIVERSITY OF CANTERBURY

CHRISTCHURCH, NEW ZEALAND

Internship

at

GNS SCIENCE

Supervisor: Phil Davies

Mentors: Dr. Robert Duxton & Dr. Alexandre Dunant

Word Count: 10129

January, 2021

DECLARATION

We, Martin Nguyen and Jian Di, declare that the report namely "Applications of generative models to enhance the performance of complex probabilistic seismic models" is completed by us. The report is based on the ideas, instructions, assistance and work experience from GNS Science and the University of Canterbury.

We also declare that this report is our original work and did not breach any existing copyright. The information contained in the report has not previously been submitted for academic examination towards any qualification.

The report has been prepared and completed for the academic purpose of the Master of Applied Data Science course – Master's Degree.

Thong Minh Nguyen (Martin)

ID: 68623131

Years: 2019 – 2021

School of Mathematics and Statistics

University of Canterbury

Jian Di

ID: 61972461

Years: 2019 – 2021

School of Mathematics and Statistics

University of Canterbury

Acknowledgement

The completion of this report required great help from various individuals. Thanks to them, the researchers achieved their goals in carrying out this project.

We, Martin Nguyen and Jian Di, the researchers of this report, would like to give a big thankyou to the following people for their assistance and advice.

Firstly, we want to express our deepest appreciation to our Project Advisor, Phil Davies, our mentors at GNS Science, Robert Buxton and Alexandre Dunant. Their instructions and suggestions contributed invaluable support to make this report utilitarian and worthwhile. The results could not be delivered sensibly and thoroughly without their help. Working with them was very interesting and we learnt a lot.

Besides this, we would like to thank all our lecturers in the Master of Applied Data Science course for the lectures and background materials distributed during the courses. Additionally, we also give thanks to the University of Canterbury staff members for helping us with the necessary information and assistance.

Last but not least, we want to express our gratitude to all our friends for their time and the best supports during our studies and project time. Especially, we are much obliged to our families, who always encouraged and assisted us with valuable ideas throughout this process.

Once again thank you everyone for helping us complete this report.

Thong Minh Nguyen (Martin) Jian Di

Executive Summary

The project aimed to develop a generative model that can learn the underlying earthquake features of tabular data to generate synthetic data. It recognized the need to reduce the complexity and computation time of OpenQuake (OQ) Engine, a well-known Probabilistic Seismic Hazard Analysis (PSHA) model.

The report outlined the processes and the results of this project, including the research about the background information of PSHA, the Generative Adversarial Network (GAN) models, and the similarity measures. Before any execution, the dataset size was reduced to a sub-dataset to use Google Colab (the free version). After that, the Conditional Tabular GAN (CTGAN) model from the 'ctgan' library version 0.2.2 with a set of optimal hyperparameters was selected. Logistic Detection, Cosine Similarity, Correlation and Mapping methods were applied to evaluate the similarity between the sub-dataset and the synthetic dataset.

The synthetic dataset produced by the selected CTGAN model was relatively similar to the training sub-dataset. However, one variable could not be learnt because it was almost constant at 0, which led to the discussion of near-zero-variance and outliers. Several modifications were tried to address this, but they were not successful. Hence, future studies would be required.

Contents

Lists of figures and tables ..... **i**

Figures i

Tables ii

Project **1**

A. Introduction **2**

1. Organisation 2

2. Overview of the project 2

3. Research aim 3

4. Research objectives 3

5. Constraints 3

6. Data 4

B. Literature review **4**

1. Background of the problems of PSHA 5

2. GAN models 6

3. Evaluation 7

C. Methodology **8**

1. Data retrieval 8

2. Data handling before modelling 9

2.1. The full dataset 9

2.2. The sub-dataset 9

4. Exploratory Data Analysis 12

5. Data pre-processing for data problems 12

6. Modelling 15

6.1. Model selection 15

6.2. CTGAN model 16

6.3. Tuning hyperparameters 18

7. Evaluation system 20
7.1. The ideas of evaluation 20
7.2.1. Evaluating on a whole dataset 21
7.2.2. Evaluating individual column 22
7.2.3. Evaluating relationships with key variable 22
7.2.4. Evaluating the patterns through maps 23
8. Trying the developed script with the full dataset 24
D. Data ethics 25
E. Results 28
1. General results before the synthetic dataset 28
1.1. Comparison between the full dataset and the sub-dataset 28
1.2. Data pre-processing 30
1.3. Selected model 32
1.4. Tuning hyperparameter 33
2. Results on evaluating synthetic dataset 35
2.1. Synthetic dataset 35
2.2. Comparison on the statistical summary 36
3. Comparison on evaluation tools 38
3.1. Logistic Detection method 38
3.2. Cosine Similarity method 38
3.3. Correlation method and Mapping method 39
3.4. Problem 42
3.5. Trying the developed script with the full dataset 42
3.6. Summary of the results 43
F. Discussion and conclusion 45
G. Further research 46
References 47
Appendix 51

Lists of figures and tables

Figures

Figure 1. Formula for objective of the Discriminator 17
Figure 2. Formula for objective of the Generator 17
Figure 3. Minimax game of the Generator and the Discriminator 17
Figure 4. Architecture of CTGAN for numeric variables 18
Figure 5. Evaluation system 21
Figure 6. The error of the 'sdv' package 22
Figure 7. Suggestion from the author of the 'sdv' package 32
Figure 8. The hyperparmeter tuning process of the batch_size 35
Figure 9. The result of the Logistic Detection 38

Tables

Table 1. Terms, Abbreviations, and explanation iii
Table 2. Stakeholders and their benefits 3
Table 3. Description of variables in the full dataset 4
Table 4. Summary of variables in the full dataset and the sub-dataset 9
Table 5. Settings to try the full dataset 10
Table 6. Data pre-processing to solve near-zero-variance and outliers 13
Table 7. Trial models 15
Table 8. Hyperparameters of the 'ctgan' package version 0.2.2 18
Table 9. Tuning hyperparameters process 19
Table 10. Evaluation ideas 20
Table 11. Data ethics 25
Table 12. Comparison between the full dataset and the sub-dataset 28
Table 13. Results of data pre-processing 30
Table 14. Results of the model selection 33
Table 15. Results of hyperparameters 34
Table 16. The synthetic dataset 36
Table 17. Synthetic dataset and comparison on statistical summary 36
Table 18. The Cosine Similarity results 38
Table 19. The Correlations scores 39
Table 20. Synthetic dataset and comparison on Correlation and Mapping methods 41
Table 21. Summary of results 43

Table 1. Terms, Abbreviations, and explanation

Terminologies	Abbreviations	Explanation
Probabilistic Seismic Hazard Analysis	PSHA	Baker (2013) explained Probabilistic Seismic Hazard Analysis "aims to quantify the uncertainties - location, size, and resulting shaking intensity of future earthquakes, and combine them to produce an explicit description of the distribution of future shaking that may occur at a site" (p.13).
OpenQuake	OQ	OpenQuake is comprised of the engine, platform and tools – caters to a variety of users, from modellers and researchers to emergency planners. OpenQuake is used for a wide range of purposes for disaster risk reduction and management (Global Earthquake Model, 2020). This model is developed based on PSHA.
OpenQuake Engine	OQ Engine	The OpenQuake Engine is a free, open-source software for the assessment of earthquake hazard and risk (Global Earthquake Model, 2020).
Generative model		Brownlee (2020) defined "generative modelling is an unsupervised learning task in artificial intelligence (AI) that involves automatically discovering and learning the regularities or patterns in input data in such as way that the model can be used to generate or output new examples that plausibly could have been drawn from the original dataset" (p.3).
Generative Adversarial Network	GAN	Brownlee (2020) explained "GAN are based on a game theoretic scenario in which the generator network must compete against an adversary. The generator network directly produces samples. Its adversary, the discriminator network, attempts to distinguish between samples drawn from the training data and samples drawn from the generator. Generator is a sub-model that is used to generate new plausible examples from the problem domain. Discriminator is a sub-model that is used to

		classify examples as real (from the domain) or fake (generated)" (p.7).
Conditional Tabular Generative Adversarial Network	CTGAN	Xu et al (2019) CTGAN is a GAN-based method to model tabular data distribution and sample rows from the distribution.
Tabular data		Dror (2020) defined "tabular data presents the data in several rows and columns where the first row/column usually represents the names of fields explaining the underlying data" (p.297).
Ground-motion prediction equations	GMPEs	A set of equations that "given some fundamental parameters characterizing the source, the propagation path and the site (in the simplest case magnitude, distance and VS,30) - computes the value GM of a (scalar) ground motion intensity parameter." (Global Earthquake Model, 2020)
Mean Square Error	MSE	Mittelhammer (2013) defined that Mean Square Error " is an alternative term for the expected squared distance between outcomes of an estimator and what it is estimating" (p.376)
Root Mean Square Error	RMSE	"The square root of MSE commonly is used to report accuracy in the same units as the original measurements" (U. S. Department of Agriculture, 2019, p. 338)
Mean Absolute Error	MAE	Commer (2020) defined that "Mean absolute error is a measure of errors between paired observations expressing the same phenomenon" (p.79)
Synthetic Data Vault	SDV	"The Synthetic Data Vault (SDV) enables end users to easily generate synthetic data synthetic data for different data modalities, including single table, relational and time series data." (MIT Data To AI Lab, n.d.)

Project

Applications of generative models to enhance the performance of complex probabilistic seismic models

A. Introduction

1. Organisation

GNS Science is a Crown Research Institute (CRI) and is New Zealand's leading geoscience and isotope research and consultancy services (GNS Science, n.d.). Their purpose is to understand natural Earth system processes and resources, then translate these into economic, environmental and social benefits to government, communities and others (GNS Science, n.d.).

The representatives of the GNS Science – Robert Buxton (AI and Advanced Analytical team) and Alexandre Dunant (Risk and Infrastructure team) have suggested the project of applying Artificial Intelligence to the PSHA modelling process to reduce the computational overhead. Two students – Martin Nguyen and Jian Di from the Masters of Applied Data Science programme at the University of Canterbury (UC) will collaborate to develop this project under the supervision of Phil Davies.

2. Overview of the project

In seismic hazard analysis, Probabilistic Seismic Hazard Analysis (PSHA) has commonly been applied to produce a clear picture of the distribution of the future quakes and shaking, which might happen at a site (Baker, 2013, p. 13). However, depending on the size of the investigation, the process of analyses can be extremely complex and computationally time-consuming (Bommer, Kontoe, & Papaspiliou, 2012, pp. 315-325). Hence, an alternative method applying Artificial Intelligence has been developed as a three-month project by the GNS team.

A generative model was developed to learn the underlying processes from a small sample (10% ~ 20%) of tabular data to generate similar outcomes. The tabular data was collected from the OpenQuake (OQ) Engine, which can implement PSHA. One of Generative Adversarial Network (GAN) methods, Conditional Tabular Generative Adversarial Network (CTGAN), was applied to learn the features from the tabular data. However, unlike other prediction problems in machine learning that used standard performance metrics to measure the accuracy such as Root Mean Square Error (RMSE) and Mean Square Error (MSE), the performance of the CTGAN model were assessed by Logistic Detection, Cosine Similarity, Pearson's and Spearman's Correlation, and Mapping methods.

Based on the above ideas, the project was complete at the stage that the CTGAN model was proved to be able to learn the underlying features of the dataset containing the

earthquake information. Also, the evaluation system was developed to evaluate the similarity of two datasets. Further studies to properly justify that the CTGAN model or other better GAN models could solve the problems of PSHA totally would be highly recommended.

The benefits to stakeholders from this research are provided in *Table 2*.

Table 2. Stakeholders and their benefits

Stakeholders		Expected benefits
Direct	Seismic Hazard and Risk scientists and seismologists (especially in GNS)	More efficient earthquake analyses in terms of complexity and time issues
Indirect	Governments and communities (recipients of modelling undertaken)	Helping inform their policies and actions according to the advice from risk scientists and seismologists
	Other industries – construction, insurance, and tourism	Less economic loss because of more efficient earthquake predictions

3. Research aim

To evaluate the feasibility and viability of employing the generative models to imitate the outputs of the earthquake modelling process to reduce the complexity, the computation time and the data handling overhead.

4. Research objectives

1. To identify any generative model that could learn the underlying features of the earthquake modelling processes.
2. To evaluate the similarity between the tabular data containing earthquake information and the output generated by the identified model with the tuned parameters.

5. Constraints

The project duration was our main concern. This stopped us from trying more models and more approaches to solving the issues that occurred in this project in general and the main problems of PSHA in particular. Also, the limited time of this project could not cover the time to improve the script on the full dataset. Hence, to save time, the use of small parts of the full dataset was chosen, because the earthquake features of the data, instead of the sample size, were the focus of this project. However, after all the

scripts were complete, we would still try with the full dataset in JupyterHub of the University of Canterbury with about 62 gigabytes (GB) of RAM. This process was not totally considered and investigated carefully due to the limited time, but some results would still contribute to further research.

6. Data

The provided dataset contained the estimates of the shaking hazard in the Hawke's Bay region. These data were produced using OQ, a well-known PSHA model. The dataset included 1927174 observations with 9 columns or 9 variables. The descriptions were described in *Table 3*.

Table 3. Description of variables in the full dataset

Variable names	Descriptions
Unamed: 0	This is identification (ID) variable for the whole dataset. The name would be renamed to 'order' later.
Event_id	ID for earthquake events
Site_id	ID for sites
Gmv_PGA	The shaking value at the site (in g or gravity)
Vs30	Shear-wave velocity from the surface to a depth of 30 meters (equivalent to a soil class)
Z2.5	Depth at which shear-wave velocity reaches 2.5 km/s
Z1.0	Depth at which shear-wave velocity reaches 1.0 km/s
Lon	Longitude (geometric variables)
Lat	Latitude (geometric variables)

The number of observations and variables in the whole dataset were reduced to generate a sub-dataset. This smaller dataset would be used instead of the entire dataset during the remainder of the project. The chosen sub-dataset had 145756 observations with 8 columns after removing the order column.

B. Literature review

The literature review consists of the background of PSHA and OQ Engine, a brief overview of GAN models and its current applications. This section indicated the gap that our project could fill. Following this direction and the secondary sources, the project aimed to examine if the application of the 'ctgan' could replace PSHA and OQ Engine, and made the process less complex and less time-consuming.

1. Background of the problems of PSHA

Baker (2013) described the complexity and time issues of undertaking PSHA as following:

> PSHA is a model with a combination of four massive mathematical computations on a huge amount of data – sources, magnitudes, distances and ground motion intensity. Especially, the modern intensity measurements had been becoming extremely complex due to the purpose of accuracy, so the time spent on having the results would be uncontrollable. (pp. 15-19, 27-32).

However, Baker did not develop a specific solution. These problems were made more apparent when the OQ Engine was used to undertake PSHA. The reason was that OQ Engine required many levels of modularity (Survey, 2019, p. 7) and a complicated preprocessing stage due to its limited functions (Pagani, et al., 2018, pp. 2-5). Both Sutherland et al. (2013) and Boore (2016) showed that PSHA needed a considerable number of potential shaking locations within the Hawke's Bay area caused by Hikurangi Subduction Zone and various Ground-motion prediction equations (GMPEs) to fulfil the computational criteria for two uncertainties, which led to an increase in the complexity and the computation time.

There has not been much previous research in this area. Briggs et al. (2017) listed numerous software applications such as NoSQL, Kibana, and QuakML to improve the process of earthquake analysis for Hikurangi Margin occurring within Hawke's Bay. Nonetheless, they did not describe in detail how these programs could address the complication and time issue. Alternative research by Alimoradi (2011) mentioned that Principal Component Analysis could solve the high-dimensional data similar to the earthquake data from Hikurangi Margin which could reduce the computation time in the pre-processing stage and some complexity in the later process. However, this method would not apply to this project due to no high-dimensional issues. As a whole, these previous studies did not develop any proper solutions for complexity and time problems.

Overall, the complexity and the time issue in earthquake analysis were flagged, and several methods were researched and developed. Nevertheless, because they did not properly address the problems, there was still a need for some research to develop an appropriate solution.

2. GAN models

A GAN was a generative model that learns the underlying features of the training data and has the ability to generate "false" data that was, in some cases, indistinguishable from the "real" data (Creswell, et al., 2018, pp. 1-2). GANs were the main focus of this project. Wang et al. (2019) applied GANs to the earthquake analysis as a replacement for OQ Engine. The approach produced high accuracy results using a GAN on earthquake data within the Oklahoma region. However, the study did not provide any specific figures for the results. Similarly, Li et al. (2020) applied a conditional GAN to earthquake data from Korea to replace OQ Engine. The results showed that 97.92% of the synthetic data was highly similar to the original data. However, the earthquake data in the mentioned research used waveform shapes. This was considered as image data instead of a tabular dataset, which was different from this project.

Some studies in different fields achieved good results for big tabular data. Nevertheless, they did not indicate if those models could be applied as the alternative to complicated and time-consuming procedures of OQ Engine. For instance, Karlsson & Sjöberg (2020) fit three self-developed GAN models, WGAN, TGAN, and CTGAN, for three financial datasets and achieved the lowest pMSEs (close to 1) when using the CTGAN model. Another research by Cote et al. (2020) developed the CTGAN with other mixed GAN (MC-WGAN-GP and MNCDP-GAN) methods on the dataset of French motor third-party liability policies. The CTGAN model also achieved low MSE (0.38), which showed a high similarity to the original data. These studies indicated that the CTGAN approach seemed to be the method that produced the most efficient results. However, they did not provide information about whether the model could address the complexity and time issues in the seismic analysis. Hence, this gap allowed this project to examine if any GAN, especially CTGAN could solve the problems of PSHA.

CTGAN and some other GAN models were built up as packages and could be applied by programming languages. The CTGAN method could be broadly applied under a Python and R package built up by a group of Massachusetts Institute of Technology (MIT) students (Xu, et al., 2019, pp. 1-3). Apart from that, the other MIT students built

up a package named Synthetic Data Vault ('sdv') contained the 'ctgan' package and models could produce a similar dataset. These models overcame some common tabular data problems such as mixed data types, non-Gaussian distributions and highly imbalanced categorical columns. These advantages would reduce the complication and the time issues at the preprocessing stage in OQ Engine. The CTGAN model was once applied as an R package in 2020 to research insurance data by Kuo Kevin. In this study,

> He used the distribution histograms and RMSE to examine the performance of CTGAN model on TPL Frequency and Shock Lapse datasets. Apart from similar plots of original and synthetic data were almost identical (pp. 1-7).

Similarly, our project would apply the 'ctgan' package and other GAN methods attached in 'sdv' library in Python to solve the complexity and time problem of PSHA in OQ Engine.

Overall, GAN methods were not researched to apply to the tabular data containing the earthquake information to replace PSHA in OQ Engine. Therefore, this project would try the 'ctgan' package and other similar approaches in the 'sdv' package in Python to generate synthetic earthquake data.

3. Evaluation

To evaluate the similarity of the real and fake or 'false' datasets, in 2019, the package 'sdv' were introduced into practice in Python. This library also included some functions to evaluate the similarity between the two datasets. However, until now, the package was still in progress of being updated. As a consequence, the research about this method was limited, which caused the low reliability. Furthermore, some functions from the 'sdv' package would only evaluate the similarity of the whole real dataset and the synthetic dataset.

Cosine Similarity, Jaccard Similarity and correlation were three other methods that were used widely. Zahrotun (2016) had practical comparative research on Cosine Similarity and Jaccard Similarity. The results indicated Cosine Similarity outperformed the Jaccard Similarity due to not requiring values of a variable to be sorted before computation. However, these methods only performed well on one variable. Therefore, Cosine Similarity would be applied to evaluate each variable of the dataset in this project.

The correlation method is used to evaluate the similarity of relationships between two items. Dharaneeshwaran et al. (2017) proved that the correlation approach performed well on the dataset with a critical variable. Also, the method was used widely by Amazon and Netflix. The correlation approach evaluated the similarity of relationships of variables due to its high reliability and efficiency in our project.

The Mapping methods compared two datasets in seismic analysis. Rondon and Banchs (2018) developed the mapping method for producing similar maps with supports from deep learning, Principal Components Analysis (PCA), and Independent Component Analysis (ICA). The Mapping method visualised the clusters of data. The similarity of the dataset's patterns could be perceived quickly through this process. This approach would evaluate the similarity in the patterns of the datasets in this project.

In summary, a new evaluation system was developed in this project based on previous studies. The measures included a function from the 'sdv' library, the Cosine Similarity method, the Correlation method, and the Mapping method.

C. Methodology

1. Data retrieval

Alexandre Dunant used the OQ Engine to produce six tables as six files of predicted tabular data containing earthquake information within Hawke's Bay area. "OQ_data__region.csv" was created by combining four these files.

2. Data handling before modelling

There were three types of datasets used in this project for different processing and CTGAN modelling steps. The full dataset was the original earthquake dataset produced from OQ Engine and loaded from the csv file. The sub-dataset was extracted from the full dataset by narrowing down the longitude and latitude, and removing the order column. The synthetic dataset was generated from the CTGAN model having the same size as the sub-dataset. The general information about the full dataset and its sub-dataset were summarised in *Table 4*.

Table 4. Summary of variables in the full dataset and the sub-dataset

Variable names	**Types**	**The full dataset** (1927173 x 9)	**The sub-dataset** (145756 x 8)
Unamed: 0	Positive integer	Renamed to 'order'	Removed
Event_id	Positive integer	✓	✓
Site_id	Positive float	✓	Changed into Positive integer
Gmv_PGA	Positive float	✓	✓
Vs30	Positive float	✓	✓
Z2.5	Positive float	✓	✓
Z1.0	Positive float	✓	✓
Lon	Positive float	✓	✓
Lat	Negative float	✓	✓

2.1. The full dataset

Removing the missing value at this step, though the 'ctgan' library could handle the missing values, was because the CTGAN model would learn the dataset without missing values to perform better. There was only one missing observation, which led to a small ratio between the missing observation and the whole number of observations (1/1927173). Therefore, removing this observation would not cause any unbiased situations or a weaker performing training set.

2.2. The sub-dataset

Many versions with different settings were tried within the 'ctgan' library (version 0.2.2) to check whether it could work in google colab. The settings were based on parameters of the 'ctgan' library including 'discrete_columns' and 'epochs'. The sample fraction and other changes were counted and described in *Table 5*.

Table 5. Settings to try the full dataset

Settings	Descriptions	Results
Setting 1	– Sample fraction = 0.1 – Removing 'order' column – Discrete_columns = ['event_id'] – Epochs = 100	The correlations of event_id, and site_id with other variables were high (all were about 0.9). The CTGAN model could not learn the boundaries of the Hawke's Bay's shape through the longitude and latitude.
Setting 2	– Sample fraction = 0.01 – Removing the 'order' column – Changing the site_id format from float to integer – Discrete_columns = ['site_id', 'event_id']	The sample fraction was set at 0.01 as a low number of observations to avoid getting crashed. The correlations of event_id, and site_id with other variables were lower than setting 1. The model could not learn the features of PGA. However, the code was crashed when the number of epochs was increased. Also, the shape of Hawke's Bay could not be learnt.
Setting 3	– Sample fraction = 0.01 – Discrete_columns = ['event_id', 'order']	The sample fraction was set at 0.01 as a low number of observations to avoid getting crashed. The code kept being crashed when the number of epochs was increased approximately over 80. Also, the result was lower than setting 2.

Setting 4	– Sample fraction = 0.01 – Removing the 'order', 'lon', 'lat' columns (the geometric variables were merged with sitemesh_133.csv later) – Converting the site_id format from float to integer – Discrete_columns = ['site_id', 'event_id'] – Epochs = 500	The sample fraction was set at 0.01 as a low number of observations to avoid getting crashed. The correlations were not as high as the setting 1, but still better than setting 2. The model could still not learn the features of PGA. However, the model could learn the shape of Hawke's Bay through the site_id.

As the code kept crashing due to the limited size of RAM of Google Colab (the free version), the full dataset was cut down to a sub-dataset to solve this problem and reduce the training time of the CTGAN model. The sub-dataset would be used throughout this project. After all the codes were developed, the full dataset was tried on this script and explained later because we still had time left.

The sub-dataset was built based on setting 4 because it could learn the shape of Hawke's Bay (see Table 5). However, there were several changes to improve the performance of the model.

At first, the full dataset was reduced by putting the boundaries on longitude and latitude instead of sampling or resampling to see clear patterns and results when mapping and evaluating in the later process. In particular, latitude was $-39.70 \leq \text{lat} \leq -39.40$ and longitude was $176.55 \leq \text{lon} \leq 177.90$. This sub-dataset was selected because it could mostly represent the whole dataset. Nevertheless, there were still some features that this sub-dataset could not include and will be discussed later.

Secondly, the sample fraction could be used between 0.1 and 0.2. In that, 0.2 was chosen because the higher number of observations in training data would help the model learn better.

Thirdly, the 'order' column was removed from the sub-dataset because it was not required in the modelling process and not produced by OQ Engine. This process would also save more RAM space in Google Colab.

Fourthly, the longitude and latitude were still be included in the sub-dataset, but would not be used in training process since the model could not learn the shape of the

Hawke's Bay area. Instead, the file 'sitemesh_133.csv' including the 'site_id', the 'lon', and the 'lat' would be merged with the dataset produced from the CTGAN model.

Fifthly, the format of the 'site_id' was converted from float to integer to have the same format when merging with the 'sitemesh_133.csv' later. Additionally, the 'site_id' should be kept as an integer as it was employed as the ID variable for both longitude and latitude.

Sixthly, the number of epochs were kept at 100 or 200 considering the learning time spent on trials. We would try with these small quantities of epochs first to see if the model could still learn well and increase the number of epochs for better performance.

As a whole, the sub-dataset of data included 145756 observations with 8 columns. Other variables containing earthquake information including event_id, site_id, gmv_PGA, Vs30, Z2.5, and Z1.0 would be trained.

4. Exploratory Data Analysis

Apart from the missing observation mentioned above, the other statistical analysis tools were also required, especially the correlation matrix heatmap was used to show the figures and the colors of strong/weak positive/negative correlations among variables in the dataset. Secondlly, the calculation results of variances of each variable in the dataset would indicate the near-zero-variance problem. Thirdly, the distributions would help to check if there was any gap in the variable. Lastly, the boxplot along with statistical summary (mean, min, max, quantitle distribution, standard deviation, and count) were used to check potential outliers in the variables. These statistical figures would also be used to measuer the similarity among the full dataset, the sub-dataset and the synthetic dataset. At some level, the CTGAN model could learn a variety of variances, distributions, outliers and other statistical features of the earthquake data.

5. Data pre-processing for data problems

The PGA had poor performance. The near-zero-variance and outliers might be the reasons for PGA cannot be learned by the CTGAN model. Since the CTGAN model from 'ctgan' package version 0.2.2 took time to be adjusted, to save time, we tried to transfer the PGA before learning, which were listed in details in *Table 6*. However, the results indicated that the CTGAN model could still not learn the features of PGA from the dataset.

Table 6. Data pre-processing to solve near-zero-variance and outliers

Methods	Descriptions	Reasons
Solving near-zero-variance		
Scale(PGA)	Using scaling functions from the scikit-learn package: – StandarScaler() – MinMaxScaler() – QuantileScaler() After the process, the PGA was kept at scaled values for model learning	Since there was no function to inverse the scaling from a new dataset, we would try this method to see if both the sub-dataset and synthetic dataset could show the same patterns as before scaling.
PGA * constant value	The constant values were: – 9.80665 (to convert from g-unit to meter/square) – 1000 – 300 The results were transformed back to their original form	To increase the variance.
(Scale(PGA) * constant value)^2	Using scaling functions from scikit-learn package: – StandarScaler() – MinMaxScaler() – QuantileScaler() The constant values were: – 9.80665 (to convert from g-unit to meter/square) – 1000 – 300 The results were transformed back to only scaled form	To increase the variance by using interaction and changing form of PGA variable to see if the model could learn when the variance was increased.
(Scale(PGA) * constant value)^3		
PGA * variable	Variables:	

PGA * variable1 * variable2	– Event_id – Site_id – Vs30 – Z2.5 – Z1.0 The results were transformed back to their original form	
PGA * variable1 * (variable2)^2		
PGA * (variable)^2		
(PGA * variable)^2		
Scale(PGA) * variable	Using scaling functions from scikit-learn package: – StandarScaler() – MinMaxScaler() – QuantileScaler() Variables: – Event_id – Site_id – Vs30 – Z2.5 The results were transformed back to only scaled form	
Scale(PGA) * (variable)^2		
(Scale(PGA) * variable)^2		
Scale(PGA) * constant * variable	Using scaling functions from scikit-learn package: – StandarScaler() – MinMaxScaler() – QuantileScaler() The constant values were: – 9.80665 (to convert from g-unit to meter/square) – 1000 – 300 Variables: – Event_id – Site_id	
Scale(PGA) * constant^2 * variable^2		
(Scale(PGA) * constant * variable)^2		

	– Vs30 – Z2.5 The results were transformed back to only scaled form	
Solving outliers		
Transformation into normal shape	Using PowerTransformer() function from the scikit-learn package: – Box-cox – Yeo-Johnson After the process, the PGA was kept at transformed values for model learning	To solve the outliers by transforming the values or removing outliers.
Removing outliers		

6. Modelling

6.1. Model selection

9 versions of GAN models and 3 versions of other models (updated to 04/01/2020) were selected due to their ability to produce a similar dataset. They were listed in *Table 7*.

Table 7. Trial models

Models	Descriptions	Main functions
GAN family		
Model 1	Separate 'ctgan' package version 0.2.2	Using CTGANSynthesizer
Model 2	Separate 'ctgan' package version 0.3.0	Using CTGANSynthesizer
Model 3	Separate 'ctgan' package version 0.3.1	Using CTGANSynthesizer
Model 4	Attached CTGAN package version 0.2.2	Using CTGAN in 'sdv' package version 0.5.0

Model 5	Attached CTGAN package version 0.3.0	Using CTGAN in 'sdv' package version 0.6.0
Model 6	Attached CTGAN package version 0.3.1	Using CTGAN in 'sdv' package version 0.6.1
Model 7	Attached CopulaGAN package	Using CopulaGAN in 'sdv' package version 0.5.0
Model 8	Attached CopulaGAN package	Using CopulaGAN in 'sdv' package version 0.6.0
Model 9	Attached CopulaGAN package	Using CopulaGAN in 'sdv' package version 0.6.1
Other models (different from GAN family)		
Model 10	Attached GuassianCopula package	Using GuassianCopula in 'sdv' package version 0.5.0
Model 11	Attached GuassianCopula package	Using GuassianCopula in 'sdv' package version 0.6.0
Model 12	Attached GuassianCopula package	Using Guassian Copula in 'sdv' package version 0.6.1

6.2. CTGAN model

The following explanation was based on the studies of Goodfellow et al. (2014), Hui (2018), Lei et al. (2019), and Kuo (2019). The CTGAN model contained two deep neural networks, the Generator G(z) and the Discriminator D(x). In that, the Generator G(z) would take some noise vector z sampled from Gaussian Mixture Model (GMM) to create an instance of a row – x (x=G(z)). In that, z presented the latent patterns of the rows, for example, the distributions. The GMM would help CTGAN to solve multi-modal distribution problems in numeric columns. At the same time, the condition vector would be created by one-hot encoding the random selection of a column and one of its levels. For example, for two discrete variables, Var_1 = {3, 4, 5} and Var_2 = {4, 5}, the condition (Var_2 = 4) was displayed by the mask vectors m_1 = [0, 0, 0] and m_2 = [1, 0]. Following this, the condition vector would be [0, 0, 0, 1, 0]. This method aimed to deal with high imbalanced discrete variables and would be applied as an input to the Generator G(z).

The Discriminator D(x) would guide the Generator G(z) on what row to create. The Discriminator D(x) was trained with real and generated rows to learn what features make the row look the same as the original. The Discriminator D(x) would also provide a scalar score representing how likely it thought the instance came from the real dataset. Next, we would discuss about the objectives of the Discriminator and the Generator.

The objective of the Discriminator was to maximize the chance to recognize the original rows as real and the generated rows as fake. The loss function would evaluate how well the Discriminator worked. Following this, the cross-entropy would be employed to measure the loss. The objective of the Discriminator could be written in *Figure 1*. V(D) denoted the value function.

$$\max_D V(D) = \underbrace{\mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})]}_{\text{recognize real rows better}} + \underbrace{\mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]}_{\text{recognize fake rows better}}$$

Figure 1. Formula for objective of the Discriminator

The objective of the Generator was to generate x (the rows) with the highest possible value of D(x) to fool the Discriminator. It could be written as below. V(G) denoted the value function.

$$\min_G V(G) = \underbrace{\mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]}_{\text{optimize G that can fool the discriminator the most}}$$

Figure 2. Formula for objective of the Generator

Based on the above formulas, CTGAN could be seen as a minimax game in which the Generator tried to minimize V while the Discriminator D tried to maximize it. In summary, the value function could be rewritten in *Figure 3*.

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))].$$

Figure 3. Minimax game of the Generator and the Discriminator

Both the Generator and the Discriminator would be trained in alternating steps until the Generator produced a good quality dataset. *Figure 4* summarized the data.

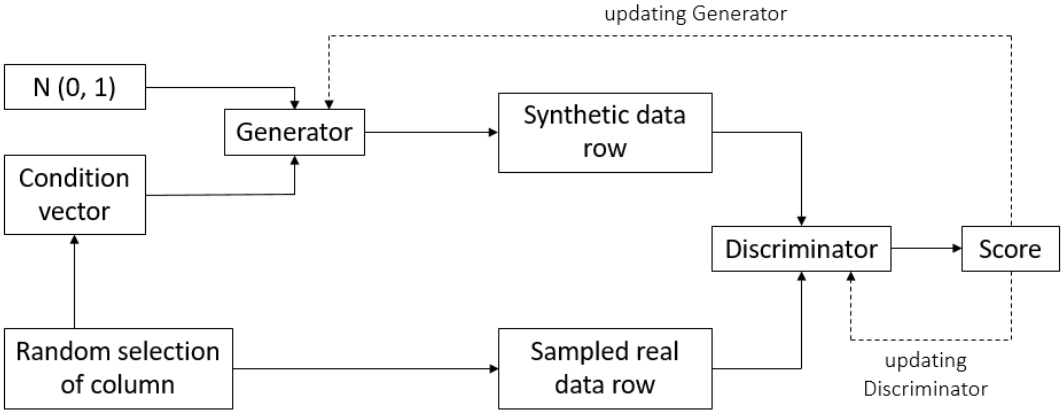


Figure 4. Architecture of CTGAN for numeric variables

6.3. Tuning hyperparameters

When trying hyperparameters with default values for all the models from 1 to 12, model 1 outperformed the others. Also, some of them apart from model 1 had errors while being trained. Additionally, the selected model would be CTGAN from the 'ctgan' package version 0.2.2 as explained later in the *Results* section. Therefore, tuning hyperparameter process of model 1 was the focus in this part.

There were 6 hyperparameters in the function 'CTGANSynthesizer' from the 'ctgan' package version 0.2.2. They were listed in *Table 8*.

Table 8. Hyperparameters of the 'ctgan' package version 0.2.2

Names of hyperparameters	Explanation	Defaults
Embedding_dim	The size of the random sample passed to the Generator	Defaults to 128
Gen_dim	The size of the output samples for each one of the Residuals. A Residual Layer would be created for each one of the values provided.	Defaults to (256, 256)
Dis_dim	The size of the output samples for each one of the Discriminator Layers. A Linear Layer will be created for each one of the values provided	Defaults to (256, 256)
l2scale	Weight decay for the Adam Optimiser	Defaults to 1e-6
Batch_size	The number of data samples to process in each step	Defaults to 500
Epochs	The number of training epochs	Defaults to 300

Different sets of hyperparameters were tested to control the learning behaviour of CTGAN and to ensure the quality of the generated data and of the processing time. We used the default values as the start point for each hyperparameter and replaced them according to the best-performing result. The list of trials and results were listed in *Table 9*.

Table 9. Tuning hyperparameters process

Hyperparameters	**Trials**
Embedding_dim	The choices were increased or decreased from the default value: 32, 64, 128, 512, 1024
Gen_dim	The choices were increased or decreased from the default value and more would be listed in an excel file: (256, 256) (256, 512) (256, 64) (256, 128, 64) (256, 256, 128) (256, 256, 64) (128, 256) (512, 256, 64)
Dis_dim	(512, 256, 64, 1024) (512, 256) (64, 256) (64, 128, 256) (1024, 512, 256) (128, 256, 256) (64, 128, 256, 256) (1024, 512, 256, 256)
l2scale	The choices were increased or decreased from the default value: 1e-4, 1e-5, 1e-6, 1e-7
Batch_size	The choices were increased or decreased from the default value: 100, 500, 1000, 2000, 2500, 3000, 3500, 4000

Epochs	The choices were increased or decreased from the default value: 50, 100, 200, 300, 500, 1000

7. Evaluation system

7.1. The ideas of evaluation

Evaluation methods of tabular data using CTGAN were open-ended questions for different datasets. Although we would have some results from comparing the statistical summary, these figures were not enough to evaluate the similarity. Therefore, we built up our similarity measures. The ideas were expressed in *Table 10*.

Table 10. Evaluation ideas

Stages	Explanations
1. Evaluating on a whole dataset	Providing the general assessment on the similarity between the sub-dataset and the synthetic dataset.
2. Evaluating column-by-column	Providing the information about how similar between values of a variable in the sub-dataset and the synthetic dataset would be.
3. Evaluating on the relationships with key variable	Providing the information that with a same value of the key variable, if a value of the given variable in the sub-dataset would be similar to that in the synthetic dataset.
4. Evaluating the patterns through mapping	Providing the assessment on the similarity of patterns of each variable in the sub-dataset and in the synthetic dataset.

7.2. Methods

The explanations about the trials and choices of methods in the following four sections represented four evaluation stages. Logistic Detection would evaluate a whole dataset. Cosine Similarity would evaluate several variables individually. The Correlation would evaluate the relationship with the key variable. Mapping would evaluate the patterns.

The evaluating process was described in *Figure 5*.

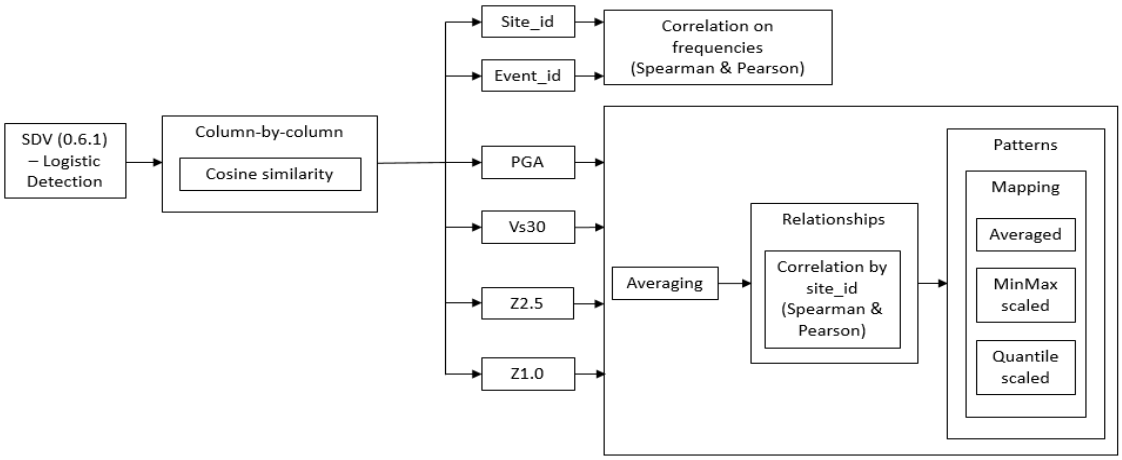


Figure 5. Evaluation system

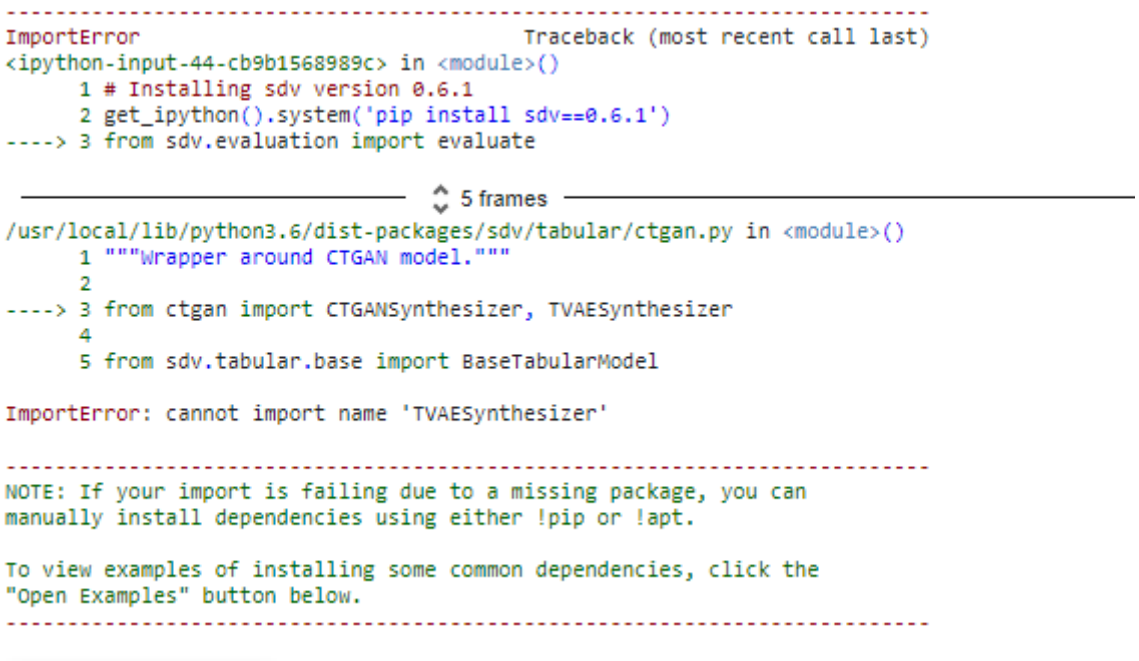
7.2.1. Evaluating on a whole dataset

We applied the metrics from the 'sdv' package version 0.6.1 to evaluate the similarity between two whole datasets. This package included 4 methods which were from 4 metric families. They were statistical metrics, likelihood metrics, machine learning efficacy metrics, and detection metrics. Logistic Detection from the detection metrics was selected because it could handle any variables, especially numeric types. It could also solve some common data problems, such as outliers and correlations.

The Logistic Detection method would shuffle the sub-dataset and synthetic dataset, and would create one more column with labels (synthetic and real). The cross-validation with the Logistic Regression from scikit-learn would predict these labels. The result was closer to 1 to represent the high similarity.

In the next section, the Logistic Detection could still be used to evaluate the similarity on column-by-column (individual column) and the relationships, but we decided not to go further with this method because it needed upgrading to fix the error (see Figure 6). The required update was caused by several uncontrollable errors and not reliable. Hence, other methods would be applied instead.

Figure 6. The error of the 'sdv' package



7.2.2. Evaluating individual column

The Cosine Similarity function from the scikit-learn package could be used to examine which column or which variable was similar to each other between two datasets due to high reliability. It calculated the "... cosine of the angle between two vectors and determines whether two vectors were pointing in roughly the same direction" (Han, Kamber, & Pei, 2012, p. 45). The result would range from 0 to 1. In that, 0 represented not similar and 1 represented entirely identical.

7.2.3. Evaluating relationships with key variable

The Correlation method was executed on discrete variables and continuous variables differently. Specifically, the method would evaluate the similarity on frequencies of discrete variables. The evaluation on the similarity of relationships with key variables was implemented on continuous variables.

For two discrete variables, the site_id and the event_id, the frequencies of each of them were counted in both the sub-dataset and the synthetic dataset. After that, we merged them by the site_id and the event_id to do correlation computation and plotting.

The site_id variable was a critical variable to evaluate the similarity on its relationships with continuous variables. After numerous trials through frequencies and correlation, the result indicated high similarity of the sit_id in both two datasets. Additionally, the site_id was the ID for geometrical variables to do the mapping. Hence, we decided to

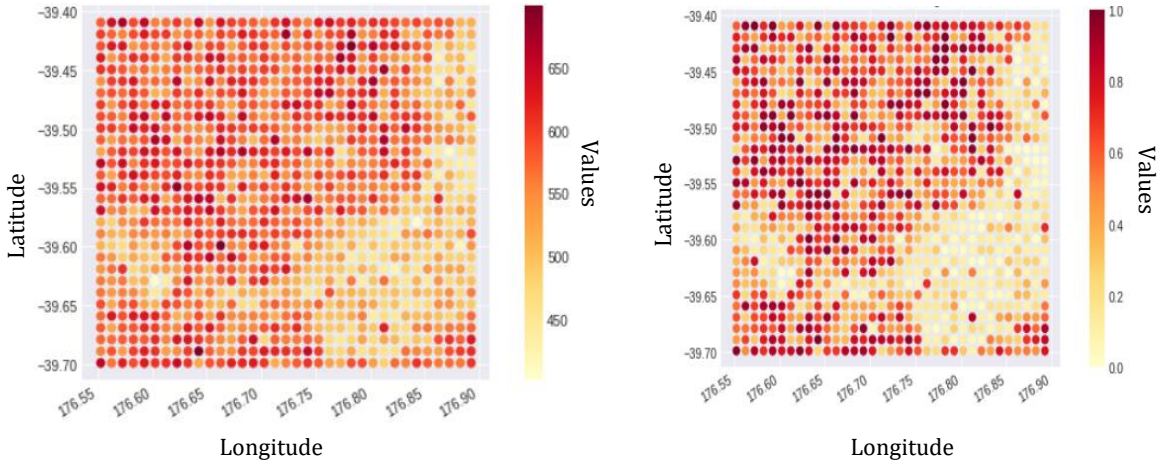
use this variable as the key variable for further similarity evaluations (Correlation and Mapping methods) with continuous variables (PGA, Vs30, Z2.5, and Z1.0).

As the information from the GNS team, the 'event_id' was not an important variable. Therefore, we would not execute further any similarity measuring on it.

For the group of continuous variables, since there were many values in each continuous variable for one site_id, all the group values were averaged based on each site_id. Although the accuracy in evaluating the similarity would be reduced due to losing values, the Correlation and Mapping methods were still good enough to evaluate the similarity on the relationships and patterns. We sorted and merged both datasets variable-by-variable by the site_id. After that, we calculated and plotted correlations to evaluate the similarity.

7.2.4. Evaluating the patterns through maps

The Mapping method checked if the CTGAN model could learn the patterns of sub-dataset. There were three types of maps for each variable. The first map used only the average values, but the features were vague. Hence, the scaling functions on each variable were applied to make the patterns clearer to observe. The maps in the *Figure 6* below would show the difference between avaraged Vs30 and scaled Vs30.



Map 1. The maps of Vs30 patterns (left – averaged Vs30, right – Quantile scaled Vs30)

Two types of scalings, 'MinMax' scaler and 'Quantile' scaler were used as functions from the scikit-learn package due to the high reliability. According to the scikit-learn package website, the 'MinMax' scaler could produce highly similar results to most of other scaling functions. Hence, it would be a good representative for them. The 'Quantile' scaler was selected due to the different results from other scaling functions.

8. Trying the developed script with the full dataset

We also ran the developed script on the full dataset. However, to avoid the crashing caused by the limited RAM in Google Colab, the script was adjusted to be run in the UC JupyterHub with about 62 GB of RAM provided by Francois Bissey. Due to the limited time left of the project, the epoch was set at 1 and 10, the sample fraction was set at 0.1, and the other hyperparameters were kept the same. This process would let us know if the script could be run on the full dataset and the results could be seen as the basis for further research.

D. Data ethics

Although the earthquake data collected from OQ Engine was the public information, we still went through some principle steps in checking the data ethics. The checklist below was built on four references. The first one was Deon, a command-line tool built and used widely to efficiently work with data scientists to add an ethics checklist to the data science projects. The other three were policies about data handling from the UC – Data Management Policy (2020), Research Conduct Policy (2020), and Storage (2020).

The explanation about the checklist would be provided in the *Appendix 24*. The information regarding each section in the checklist was described in *Table 11*.

Table 11. Data ethics

Criteria		Information
A. Data Collection	A.1. Informed consent	As the dataset was not relevant to demography and human, this section was not applicable.
	A.2. Collection bias	
	A.3. Limit PII exposure	
	A.4. Downstream bias mitigation	
B. Data Storage	B.1. Data security	All of the datasets received from GNS team (extracted from OQ Engine) were stored in personal Dropbox. GNS team, our Project Advisor, and we were allowed to access to the datasets. As the agreement from the GNS team, the datasets would be transferred to be stored in the University Dropbox. Following this, the use and sharing of these datasets would still comply with UC Research Conduct Policy (2020).
	B.2. Right to be forgotten	When the dataset stored in University Dropbox, the deletion and recovery would be more flexible. The interval between the deletion and recovery would be up to 120 days. These actions could be following through the support with ICT system team. Therefore, if the GNS team or other authorised

		parties required the datasets to be deleted or recovered, the operation would be convenient.
	B.3. Data retention plan	As the agreement with GNS team, if there were not any issues regarding accessing to the datasets and the codes during the time we were doing the project, they would be stored up to five years (following Human Ethics Committee), when we would leave the school. However, following the UC Research Conduct Policy (2020), we would still be responsible for the allowance to develop the project.
C. Analysis	C.1. Missing perspectives	The information from the datasets was not either relevant to society or any serious matters.
	C.2. Dataset bias	The possible souces of bias were not found from the datasets.
	C.3. Honest representation	All the tools to visualise, to do summary statistics, and to generate the reports were designed honestly with full references.
	C.4. Privacy in analysis	The datasets were not relevant to personal identifiable information.
	C.5. Auditability	The process of generating the ananlysis was adequately documented and could be reproducible conveniently in the future.
D. Modeling	D.1. Proxy discrimination	The CTGAN model did not rely on any specific variable that was unfairly discriminatory.
	D.2. Fairness across groups	The datasets and the CTGAN model did not include any categorical variables. They were also not relevant to any serious groups.
	D.3. Metric selection	The effects of optimising for metrics were under control of the quantity of hyperparameters provided constantly in the 'ctgan' package.
	D.4. Explainability	The explanation for the earthquake and CTGAN model terms were adequately justified and provided.

	D.5. Communicate bias	All the issues appeared during the development of the project were reported adequately to GNS team and the Project Advisor.
E. Deployment	E.1. Redress	The project process did not include any harmful stages.
	E.2. Roll back	The research of the project was mostly built based on the codes and Google Colab. Hence, these codes would be easily accessed, turned of, restarted and adjusted if necessary.
	E.3. Concept drift	As the results of CTGAN model would be slightly different every time the new samples were produced, and the random.seed() function could not be used, we stored each version of samples in a folder in Google Drive for later usage.
	E.4. Unintended use	As the notice from the beginning of the project and the script report, the version of the codes might only be applied for this type of dataset. For the other similar datasets, the codes should be adjusted.

E. Results

This section was divided into three parts. The first part included the results of the comparison between the full dataset and the sub-dataset, the data preprocessing, the model selection and the hyperparameters tuning. The second part mentioned about the synthetic data, the comparison between it and the sub-dataset based on EDA. The third part focused on the results of the evaluation system. All the saved CTGAN models and their results were stored under a folder named 'for_Martin_and_Jian'.

1. General results before the synthetic dataset

1.1. Comparison between the full dataset and the sub-dataset

The comparison of the full dataset and its sub-dataset showed how the sub-dataset could represent the entire dataset. The details were described in *Table 12*.

Table 12. Comparison between the full dataset and the sub-dataset

Methods	Full dataset	Sub-dataset	Comments
The results below could be referred to: – Correlations: The Appendix 2 and 8 – Variances: The Appendix 3 and 9 – Distributions: The Appendix 4 and 10 – Stastical summary: The Appendix 5 and 11 – Boxplots: The Appendix 6 (or 7) and 12 (or 13)			
Correlations	– Strong negative correlation between 'site_id' and 'lat' – Strong negative correlation between 'Vs30' and 'Z2.5' – Strong negative correlation between 'Vs30' and 'Z1.0' – Strong positive correlation between 'Z2.5' and 'Z1.0' (As the 'order' column were removed in the sub-	– Strong negative correlation between 'site_id' and 'lat' – Strong negative correlation between 'Vs30' and 'Z2.5' – Strong negative correlation between 'Vs30' and 'Z1.0' – Strong positive correlation between 'Z2.5' and 'Z1.0'	Highly similar

	dataset, the correlation between it and 'event_id' would not be mentioned)		
Variances	– Extremely high variances: 'event_id', 'site_id', 'Vs30', and 'Z1.0' – Extremely low variances: 'PGA' (very close to 0) and 'Z2.5' (As 'lon' and 'lat' would not be used in the modelling process, their variances would not be mentioned)	– Extremely high variances: 'event_id', 'site_id', 'Vs30', and 'Z1.0' – Extremely low variances: 'PGA' (very close to 0) and 'Z2.5'	Highly similar
Distributions	– Event_id: The distribution was skewed to the left. – Site_id: The middle of distribution was low. – Gmv_PGA: The distribution was mostly around 0 – Vs30: The distribution was mostly negative skew. The other side of distribution was also a little bit high. – Z2.5: The distribution was mostly positive skew. The other side of distribution was also a little bit high. – Z1.0: The distribution was highly more	– Event_id: The distribution was skewed to the left. – Site_id: The middle of distribution was higher. – Gmv_PGA: The distribution was around 0 – Vs30: The distribution was skewed to both side – Z2.5: The distribution was mostly skewed to both side. – Z1.0: The distribution was mostly skewed to both side.	Relatively similar. Apart from the similar points, in the sub-dataset, most of the variables seemed to have one more tail in the distributions (not only one tail as in the full dataset)

	skewed to the right than to the left.		
Descriptive summary and boxplots	– All values of all variables were larger than 0 – Site_id: This variable had the largest range of values – PGA values seemed to be very close to 0 – When the whiskers were changed from 1.5 to 5, PGA, 'Z2.5', 'Z1.0' seemed to have outliers.	– All values of all variables were larger than 0 – Site_id: This variable seemed to have the largest range of values, but it was lower than the one in the full dataset. – PGA values seemed to be very close to 0 – When the whiskers were changed from 1.5 to 5, only PGA seemed to have outliers.	Relatively similar. Apart from the similar points, the outliers in Z2.5 and Z1.0 were not presented in the sub-dataset.

This sub-dataset had almost the features of the full dataset, except for differences in distributions (Vs30, Z2.5, and Z1.0), and outliers (Z2.5 and Z1.0). Additionally, this sub-dataset also had clear patterns when mapping for easily checking the learning ability of the CTGAN model. Generally, the results from this section proved that this sub-dataset could represent the full dataset.

1.2. Data pre-processing

The values of PGA was close to 0, which caused the near-zero-variance issue and outliers. It led to a failure in the learning of the CTGAN model. The considered solution was to increase the variance or to remove the outliers. The results of data preprocessing were described in *Table 13*.

Table 13. Results of data pre-processing

Methods	Results
Solving near-zero-variance	

Scale(PGA)	After being scaled, the PGA in the sub-dataset dataset was still able to show the same patterns as when the variable was not scaled. Based on this sign, we could implement some more methods with scaling on the PGA variable.
PGA * constant value	These methods did not either make any difference in variance or in outliers of PGA. Hence, the CTGAN model was still not able to learn the features. However, this method would be combined with other methods to increase the variance of PGA.
(Scale(PGA) * constant value)^2	The results were still poor and did not indicate any sign that the CTGAN model could learn at least something.
(Scale(PGA) * constant value)^3	
PGA * variable	
PGA * variable1 * variable^2	
PGA * variable1 * (variable2)^2	
PGA * (variable)^2	
(PGA * variable)^2	
Scale(PGA) * variable	
Scale(PGA) * (variable)^2	
(Scale(PGA) * variable)^2	
Scale(PGA) * constant * variable	
Scale(PGA) * constant^2 * variable^2	
(Scale(PGA) * constant * variable)^2	
Solving outliers	
Transformation into normal shape	

Removing outliers	The results were still poor and did not indicate any sign that the CTGAN model could learn at least something.

In conclusion, the PGA variable could not be learnt by the CTGAN model from the 'ctgan' package version 0.2.2 base on these trials. The authors of the 'sdv' package suggested we try the CopulaGAN function due to being better at learning the distributions of variables. The suggestion was shown in the *Figure 7*.

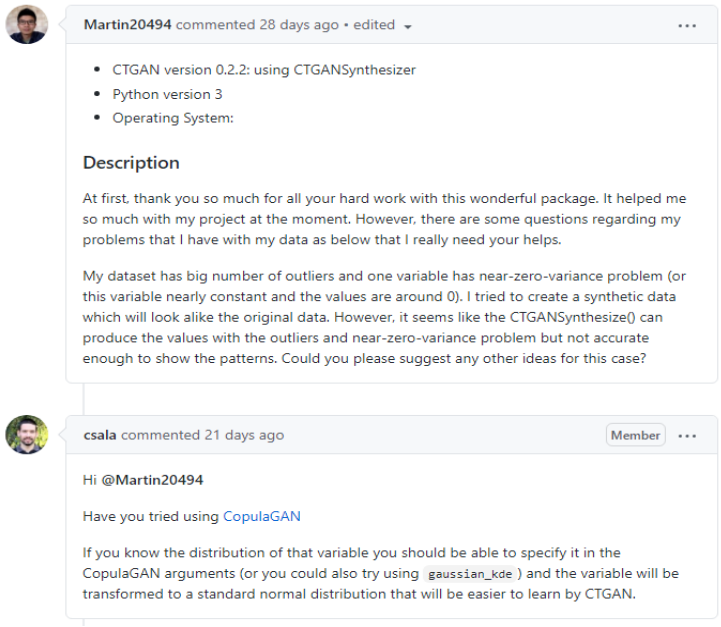


Figure 7. Suggestion from the author of the 'sdv' package

However, the results were even worse as it could not learn the 'event_id'. However, Project Advisor suggested us to transfer PGA into square-root(log-transforming('Z1.0')/PGA). The patterns of PGA behaved well after learning when mapping. It increased the variances and the range of values as the outliers disappeared, which would help the CTGAN model learn the PGA values. However, when we inversed the PGA to the original form in the synthetic dataset, the CTGAN model could still not learn it.

1.3. Selected model

The results of the model selection process were described in the *Table 14*.

Table 14. Results of the model selection

<table>
<tr><th>Models</th><th>Results</th></tr>
<tr><td colspan="2">GAN family</td></tr>
<tr><td>Model 1</td><td>– Could learn Vs30, Z2.5, and Z1.0
– Could not learn PGA</td></tr>
<tr><td>Model 2</td><td rowspan="2">– Model 2 and 3 could not learn the features of ‘site_id’. In particular, they could learn that ‘site_id’ should be equal or larger than 0.
– Could learn Vs30, Z2.5, and Z1.0
– Could not learn PGA</td></tr>
<tr><td>Model 3</td></tr>
<tr><td>Model 4</td><td rowspan="5">– Could not learn the features of ‘event_id’ and the counts of ‘event_id’
– Would get errors after several trials
– Could learn Vs30, Z2.5, and Z1.0
– Could not learn PGA</td></tr>
<tr><td>Model 5</td></tr>
<tr><td>Model 6</td></tr>
<tr><td>Model 7</td></tr>
<tr><td>Model 8</td></tr>
<tr><td>Model 9</td><td></td></tr>
<tr><td colspan="2">Other models (different from GAN family)</td></tr>
<tr><td>Model 10</td><td rowspan="3">– Could not learn the features of ‘event_id’ and the counts of ‘event_id’
– Would get errors after several trials
– Could learn Vs30, Z2.5, and Z1.0
– Could not learn PGA</td></tr>
<tr><td>Model 11</td></tr>
<tr><td>Model 12</td></tr>
</table>

As the results, the performance of model 1 was better than the others. Apart from that, the separate ‘ctgan’ package versions 0.3.0 and 0.3.1, and the ‘sdv’ package versions 0.5.0, 0.6.0, and 0.6.1 were not stable as they were unpredictable and were being updated frequently (newest update on 31/12/2020). This caused inconveniences in processing the code, time-consuming problems, and confusion for the system. Hence, we selected model 1 to check how was the learning ability of the CTGAN model. The other models should be used after the complete upgrading process from the authors, and would be implemented outside of this project.

1.4. Tuning hyperparameter

The results of tuning hyperparameters were described in *Table 15*.

Table 15. Results of hyperparameters

Hyperparameters	Results
Embedding_dim	When embedding_dim equaled to 512, the correlation scores and others were better than other choices. We chose this number for embedding_dim to work with other hyperparameters.
Gen_dim	When both gen_dim and dis_dim equaled to (512, 256, 64), the correlation scores and others were better than other choices. We chose this dimensions for gen_dim and dis_dim to work with other hyperparameters.
Dis_dim	
l2scale	When l2scale equaled to 1e-4, the correlation scores, time and others were better than other choices. We chose this l2scale to work with other hyperparameters.
Batch_size	When batch_size equaled to 3000, the correlation scores, time and others were better than other choices. We chose this batch_size to work with other hyperparameters.
Epochs	When epochs equaled to 200, the correlation scores, time and others were better than other choices. We chose this epochs to work with other hyperparameters.

As notice from the trials and the results, the values of the hyperparameters could only be increased and decreased to a specific number for each hyperparameter. For example, in the Figure 7, if we increased the batch_size over 3000 or reduced below 3000, the correlation scores and other scores would worsen, related to overfitting and underfitting issues. Please see more information in the file 'hyperparameter table' in folder 'CTGAN model'.

							Event_id	0.83656	0.954252	0.956741		CPU times: user
							Site_id	0.99513	0.604147	0.608605		52min 2s, sys:
							PGA	0.69655	0.007475	0.023955		13min 18s, total:
							Vs30	0.74672	0.074328	0.057535		1h 5min 20s
							Z2.5	0.66742	0.043862	0.021767	Loss G: -0.0002,	Wall time: 1h
512	(512, 256, 64)	(512, 256, 64)	1.00E-04	100	300	0.952371	Z1.0	0.53282	0.062077	0.021785	Loss D: -0.1279	5min 59s
							Event_id	0.83906	0.953513	0.953883		CPU times: user
							Site_id	0.99513	0.629965	0.630980		40min, sys: 5min
							PGA	0.69554	-0.003580	-0.016254		24s, total: 45min
							Vs30	0.75172	0.084373	0.070556		24s
							Z2.5	0.66617	0.115004	0.093394	Loss G: 0.1184,	Wall time: 45min
512	(512, 256, 64)	(512, 256, 64)	1.00E-04	100	300	0.938242	Z1.0	0.53900	0.117646	0.093622	Loss D: -0.3509	28s
							Event_id	0.84323	0.945971	0.962782		CPU times: user
							Site_id	0.99510	0.670193	0.677661		10min 50s, sys:
							PGA	0.68424	-0.054068	-0.068455		1min 49s, total:
							Vs30	0.75122	0.357137	0.343527		12min 40s
							Z2.5	0.66759	0.403637	0.340152	Loss G: -0.0515,	Wall time: 12min
512	(512, 256, 64)	(512, 256, 64)	1.00E-04	1000	300	0.903556	Z1.0	0.54195	0.393363	0.340973	Loss D: -0.1595	33s
							Event_id	0.84611	0.961871	0.980605		CPU times: user
							Site_id	0.99510	0.722457	0.739723		9min 30s, sys:
							PGA	0.67969	-0.125001	-0.118146		1min 43s, total:
							Vs30	0.74812	0.618057	0.609546		11min 13s
							Z2.5	0.66760	0.636918	0.562361	Loss G: -0.0887,	Wall time: 10min
512	(512, 256, 64)	(512, 256, 64)	1.00E-04	2000	300	1	Z1.0	0.53911	0.635968	0.599172	Loss D: -0.0345	59s
							Event_id	0.84550	0.971690	0.976690		CPU times: user
							Site_id	0.99513	0.705726	0.733868		8min 57s, sys:
							PGA	0.68275	-0.056817	-0.052209		1min 45s, total:
							Vs30	0.74772	0.684491	0.642558		10min 42s
							Z2.5	0.67270	0.711626	0.566366	Loss G: -0.1967,	Wall time: 10min
512	(512, 256, 64)	(512, 256, 64)	1.00E-04	2500	300	0.791667	Z1.0	0.55337	0.708193	0.598247	Loss D: -0.0451	28s
							Event_id	0.84643	0.971608	0.983124		CPU times: user
							Site_id	0.99512	0.742359	0.754021		8min 50s, sys:
							PGA	0.67973	-0.044859	-0.035287		1min 52s, total:
							Vs30	0.74869	0.748288	0.734178		10min 42s
							Z2.5	0.66840	0.760901	0.658983	Loss G: -0.2304,	Wall time: 10min
512	(512, 256, 64)	(512, 256, 64)	1.00E-04	3000	300	0.927662	Z1.0	0.53299	0.752374	0.688062	Loss D: -0.0073	27s
							Event_id	0.84559	0.968069	0.980953		CPU times: user
							Site_id	0.99513	0.720202	0.744403		9min 3s, sys: 1min
							PGA	0.68332	-0.013021	-0.021780		54s, total: 10min
							Vs30	0.74834	0.729815	0.713726		58s
							Z2.5	0.65425	0.738709	0.629832	Loss G: -0.0147,	Wall time: 10min
512	(512, 256, 64)	(512, 256, 64)	1.00E-04	3500	300	0.841025	Z1.0	0.54460	0.742980	0.683088	Loss D: -0.0367	43s
							Event_id	0.84579	0.976549	0.981093		
							Site_id	0.99515	0.750137	0.766605		CPU times: user
							PGA	0.66747	-0.062690	-0.072371		9min 4s, sys: 1min
							Vs30	0.74892	0.764802	0.744898		59s, total: 11min
							Z2.5	0.67925	0.747279	0.685131	Loss G: -0.1734,	4s Wall time:
512	(512, 256, 64)	(512, 256, 64)	1.00E-04	4000	300	0.540361	Z1.0	0.55962	0.762194	0.732078	Loss D: -0.0746	10min 49s

Figure 8. The hyperparmeter tuning process of the batch_size

Besides, the training time would be increased with the values of hyperparameters for l2scale, batch_size, and epochs. We selected the one that was good at both results and required a short period. The suitable set of values of hyperparameters were:

- Sample fraction=0.2
- Embedding=512
- Gen_dim=(512, 256, 64)
- Dis_dim=(512, 256, 64)
- l2scale=1e-4
- Batch_size=3000
- Epochs=200

2. Results on evaluating synthetic dataset

2.1. Synthetic dataset

The synthetic dataset was saved under parquet files after being produced from the CTGAN model. The parquet files were chosen due to being good at handling the big size dataset. This process would also help call this specific dataset again if necessary and avoid the random sampling of the model again. After that, the dataset was gathered into a table with longitude and latitude from 'sitemesh_133.csv' by the merge function.

Following this, the synthetic dataset contained 145756 observations with 8 variables. Both the event_id and site_id columns were an integer, the group of continuous variables, PGA, Vs30, Z2.5 and Z1.0 were all floats. The lon and lat columns were still floats as merged from the original file.

Table 16. The synthetic dataset

Variable names	Types	Synthetic data (145756 x 8)
Event_id	Positive integer	✓
Site_id	Positive integer	✓
Gmv_PGA	Positive float	✓
Vs30	Positive float	✓
Z2.5	Positive float	✓
Z1.0	Positive float	✓
Lon	Positive float	Not trained by CTGAN
Lat	Negative float	

2.2. Comparison on the statistical summary

The statistical summary of the synthetic dataset comparing with the sub-dataset was described in *Table 17*:

Table 17. Synthetic dataset and comparison on statistical summary

Methods	Synthetic dataset	Comparing with the sub-dataset
The results below could be referred to: – Correlations: The Appendix 8 and 14 – Variances: The Appendix 9 and 15 – Distributions: The Appendix 10 and 16 – Stastical summary: The Appendix 11 and 17 – Boxplots: The Appendix 12 (or 13) and 18 (or 19)		

Correlations	– Strong negative correlation between 'site_id' and 'lat'. – Strong negative correlation between 'Vs30' and 'Z2.5' – Strong negative correlation between 'Vs30' and 'Z1.0' – Strong positive correlation between 'Z2.5' and 'Z1.0'	Highly similar
Variances	– Extremely high variances: 'event_id', 'site_id', 'Vs30', and 'Z1.0' – Extremely low variances: 'PGA' (very close to 0) and 'Z2.5'	Highly similar
Distributions	– Event_id: The distribution was skewed to the left. – Site_id: The middle of distribution was a little bit high. – Gmv_PGA: The distribution was around 0 but a little wider. – Vs30: The distribution was skewed to both side – Z2.5: The distribution was mostly skewed to both side. – Z1.0: The distribution was mostly skewed to both side.	Highly similar. However, the PGA in the synthetic dataset had a bit wider distributions than in the sub-dataset.
Descriptive summary and boxplots	– All the values of all variables were positive – Site_id: This variable seemed to have the largest range of values – PGA values seemed to be very close to 0. Also, this variable seemed to have wider range of values. – When the whiskers were changed from 1.5 to 5, only PGA seemed to have outliers. However, the range of values of these outliers seemed to be shorter.	Highly similar except for PGA variable. The PGA variable seemed to have a wider range of values in the synthetic dataset than in the sub-dataset. However, the range of outliers seemed to be shorter in the synthetic dataset than in the sub-dataset.

Please from the *Appendix 14* to *Appendix 19* for more information. The CTGAN model seemed to produce a highly similar dataset. However, although the PGA variable appeared to have the same features about outliers and near-zero-variance, their range was different. The outliers had a shorter range and the typical values had a longer coverage.

3. Comparison on evaluation tools

This section was divided into four parts – the Logistic Detection method, the Cosine Similarity method, the Correlation method, and the Mapping method.

3.1. Logistic Detection method

The result of Logistic Detection was over 0.5, representing that the synthetic data might be relatively similar to the sub-dataset. However, without any previous test and research for this new package, the result would not be highly reliable. Additionally, it was not clear which variables could be learnt by the CTGAN model. The result was shown in the *Figure 6*.

Figure 9. The result of the Logistic Detection

	metric	name	score	min_value	max_value	goal
0	LogisticDetection	LogisticRegression Detection	0.54165	0.0	1.0	MAXIMIZE

3.2. Cosine Similarity method

Cosine Similarity measured the similarity between two variables. All the variables had the Cosine Similarity score over 0.5. This might be seen as a good learning from the CTGAN model. The 'site_id' variable had the highest score, which helped to support using the 'site_id' as a key variable. PGA, Z2.5, and Z1.0 had the results close to 0.5 and lower than other Cosine Similarity scores.

However, this method did not ensure about the similarity about the relationships of the key variable with other variables. Hence, the results from Correlation and Mapping method was more reliable and would show more accurate information about the similarity between two datasets. The *Table 18* showed the results of Cosine Similarity.

Table 18. The Cosine Similarity results

Variable names	Cosine Similarity
Event_id	0.84644
Site_id	0.99512
Gmv_PGA	0.67116
Vs30	0.75188

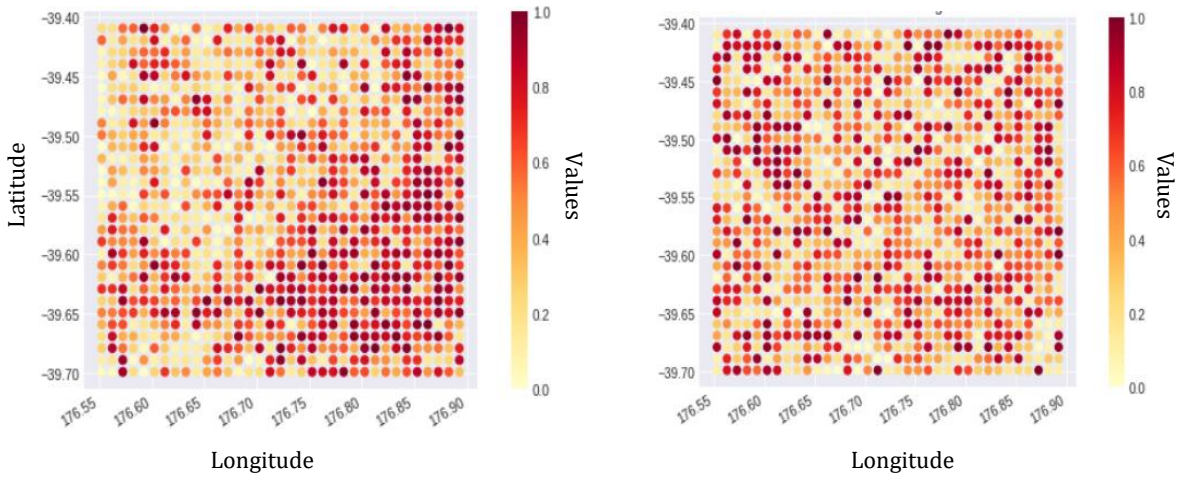
Z2.5	0.65383
Z1.0	0.52338

3.3. Correlation method and Mapping method

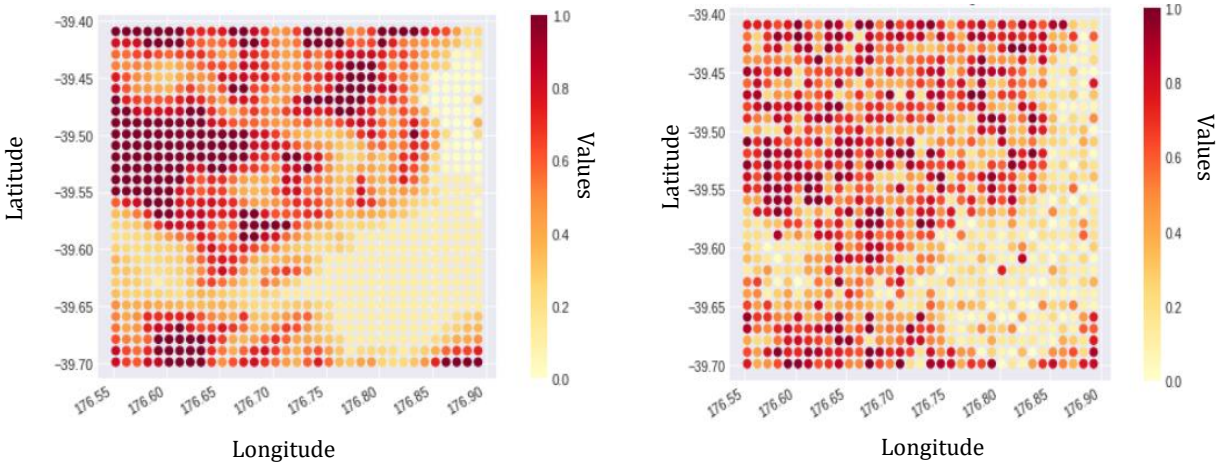
We used both two styles of Correlation – Pearson and Spearman. The reason was that columns containing comparatively large values than other columns would significantly impact the Pearson. Outliers would affect Spearman correlation coefficient. Hence, using both correlations at the same time would give us clearer and more accurate results. In the Mapping method section, the results would be shown by distributions and maps. The *Table 19* showed the results of Correlation. The maps showed the patterns of only Quantile scaled data due to the clear visions. The distributions when scaling of the averaged sub-dataset and the synthetic dataset were displayed from *Appendix 20* to *Appendix 23.*

Table 19. The Correlations scores

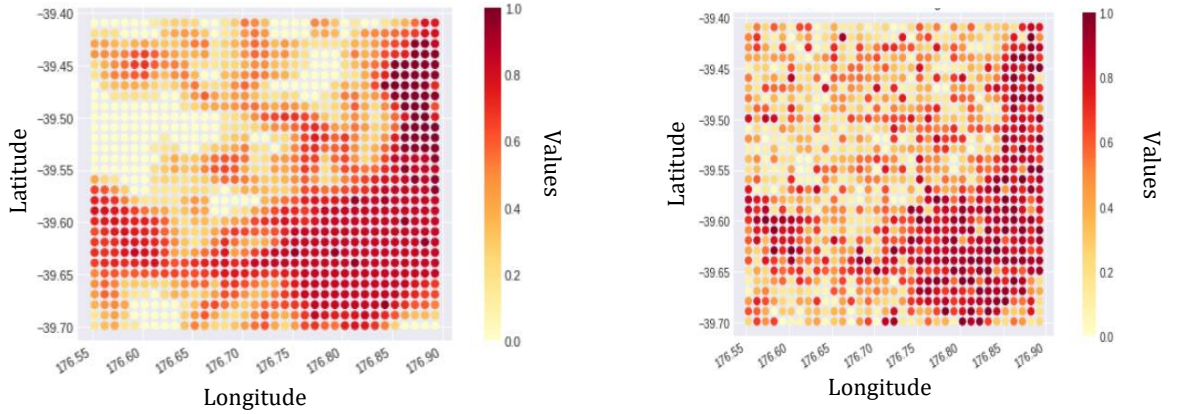
Variable names	Pearson's Correlation	Spearman's Correlation
Event_id	0.97485	0.981673
Site_id	0.73524	0.746944
Gmv_PGA	-0.029350	-0.014253
Vs30	0.712146	0.688790
Z2.5	0.737243	0.657388
Z1.0	0.717711	0.657116



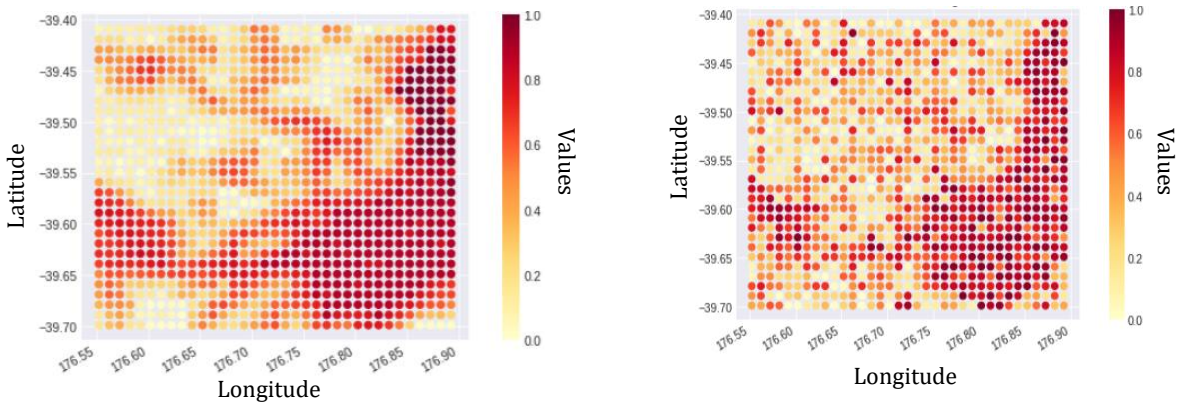
Map 2. The maps of Quantile scaled PGA (left - sub-dataset, right - synthetic dataset)



Map 3. The maps of Quantile scaled Vs30 (left - sub-dataset, right - synthetic dataset)



Map 4. The maps of Quantile scaled Z2.5 (left - sub-dataset, right - synthetic dataset)



Map 5. The maps of Quantile scaled Z1.0 (left - sub-dataset, right - synthetic dataset)

The combination of those results and explanation of Correlation and Mapping methods were summarise in *Table 20*.

Table 20. Synthetic dataset and comparison on Correlation and Mapping methods

Variables	**Correlation**	**Mapping**		**Results**
		Distributions	**Maps**	
Event_id	Highly similar	Not applicable		The results from the correlation scores were extremely high. This fits the results of the variables' distributions.
Site_id	Highly similar			
PGA	Not similar	Highly similar	Not similar	The result from the correlation score was extremely low, close to 0. The output from mapping was also poor. The patterns (the dark and light dots) were dispersed and did not form into any clusters. However, the distributions in the averaged sub-dataset and the averaged synthetic dataset were similar to each other. In particular, the distributions of PGA were mostly symmetrical in both datasets.
Vs30	Relatively similar	Not similar	Relatively similar	The results from the correlation scores were relatively high (around 0.7). The patterns (the dark and light dots) in the maps were learnt relatively well.
Z2.5	Relatively similar	Not similar	Relatively similar	

Z1.0	Relatively similar	Not similar	Relatively similar	However, the distributions in the averaged sub-dataset and in the averaged synthetic dataset were not similar. In particular, the distributions of 'Vs30', 'Z2.5', and 'Z1.0' were mostly skewed to both left and right in the averaged sub-dataset, but they were nearly symmetrical in the averaged ctgan dataset.

As to the discrete variables, both the 'event_id' and the 'site_id' were learnt well by the model. As to the continuous variables, except PGA, the other variables were relatively learnt well by the model. Besides, the results of Correlation and Mapping methods matched well each other. They relatively matched well with the results of the Cosine Similarity method (the results of PGA, Z2.5, and Z1.0 did not highly match). This meant that the values of the datasets might not be totally the same, the model could still learn well the underlying features.

3.4. Problem

The PGA had a significant problem as the CTGAN model could not learn this variable according to the result, though several methods had been tried to solve this problem. The PGA variable was extremely sensitive to a minor change in the values due to its values were constant and close to 0, which caused the near-zero-variance and outliers when there were considerable changes in the values. Hence, PGA required a high accuracy in learning to produce a similar set of values. However, the CTGAN model allowed the large differences in values and was not strict on variables sensitive to minor change. Consequently, it could produce a variable with near-zero-variance and outliers, but the accuracy of the values was extremely low.

3.5. Trying the developed script with the full dataset

Please see from the *Appendix 24* to *Appendix 28* for more information.

The results was not as good as the results of the sub-dataset, especially for the continuous variables. This could be due to the small number of epochs or the heavy outliers appeared in all continuous variables. Increasing the number of epochs, the results might be considerably better. However, due to the limited time of the project, the higher epochs would be tried in further studies. Owing to not properly investigating the developed script with the full dataset, the results of this part would

not be the focus for this project but could be seen a contribution or a basis for any later research to solve the problems of PSHA totally.

3.6. Summary of the results

Although we chose the sub-dataset could represent the whole dataset comparatively well, it still lacked some features such as outliers. Therefore, it would not be a perfect replacement for the entire dataset. Hence, the results might be slightly different according to the choice of the whole dataset or other sub-datasets. The function 'CTGANSynthesizer' from the 'ctgan' package version 0.2.2 was selected. The optimal hyperparameters were:

- Sample fraction=0.2
- Embedding=512
- Gen_dim=(512, 256, 64)
- Dis_dim=(512, 256, 64)
- l2scale=1e-4
- Batch_size=3000
- Epochs=200

The combination of the Correlation and the Mapping showed the relative similarity between the sub-dataset and the synthetic dataset. The results from these methods were more reliable than the others. The details of the results were summarised about the learning ability of the CTGAN model in *Table 19*:

Table 21. Summary of results

Did learn well	Did not learn well
– Learnt well about the size an types of the dataset (positive integers and float) – Learnt well on event_id, site_id, Vs30, Z2.5, and Z1.0 – Learnt positive integers better than positive floats	– Not able to learn PGA. In particular, not able to learn the accurate values of variable which was almost constant and the values were close to 0. This led to the wrong values of near-zero-variance and outliers

– Learnt well correlations between variables, variances, and ranges of values of variables – Learnt relatively well on distributions. As to discrete variables, the model learnt well. As to the continuous variables, if the original distributions were mostly skewed to both of the sides, the distributions of the synthetic variables might not be similar. However, the values and patterns of these continuous variables in both datasets would be highly similar. – Learnt well about having near-zero-variance and outliers in a variable – PGA. In other words, when plotting, these features were still seen in the synthetic dataset.	– Not able to learn the boundaries of shape of Hawke's Bay through longitude and latitude.

F. Discussion and conclusion

In the short period of the project, we had specific progress about testing the new concept that generative models could be applied to improve the earthquake modelling processes and document all the trials for the possible models and their results after the evaluation.

As far, we proved that the CTGAN model from the 'ctgan' package version 0.2.2 could learn most of the underlying earthquake features. However, the project results might be only correct for this specific sub-dataset by running the code on Google Colab (the free version). CTGAN learnt well on almost variables, but could not deal with the PGA column with nearly constant and zero values.

An evaluation system with a combination of the Logistic Detection, the Cosine Similarity, the Correlation, and the Mapping methods was developed to test the similarity between the two datasets, which was a massive challenge. These methods ensured the relatively similarity on the individual column, their relationships and the patterns of the two datasets.

Further research was highly recommended to complete solving the problems of complexity and time issue in OQ Engine.

G. Further research

The improvement of the codes on other sub-datasets or the whole dataset could be implemented with Google Colab Pro or with other Machine Learning platforms to extend this research. Also, another possible research direction was to solve the PGA problem. The additional function was required to be updated or added into the current 'ctgan' package that could handle constant variables or variables with near-zero-variance issue and outliers. Alternatively, a new 'ctgan' package could be developed with functions to solve this issue. Consequently, our project and the comments from users of 'ctgan' package and 'sdv' package in the SVD community forum would support their developments.

The novel concept of this project could also apply to other hazard analysis. For example, to generate a synthetic predicted dataset in flooding or volcano analyses to reduce the complexity and computation time.

References

Baker, J. W. (2013). *Introduction to Probabilistic Seismic Hazard Analysis.* Apache License.

Banchs, R., & Rondon, O. (n.d.). Seismic similarity analysis through self-organising maps. *Exploration geophysics, 38*(3), 184-188. doi:https://doi.org/10.1071/EG07018

Bommer, J. J., Kontoe, S., & Papaspiliou, M. (2012, November). An exploration of incorporating site repsonse into PSHA. *Sensitivity of Hazard Estimates to Site Response Approaches, 42*, pp. 316-330. doi:https://doi.org/10.1016/j.soildyn.2012.05.001

Boore, D. (2016). *GMPE: Past, Present and Future.* New Zealand: New Zealand Society for Earthquake Engineering.

Briggs, R., Zellman, M., Christine, P., Trainor-Guitton, W., Dashti, S., Aster, R., & Hayes, G. (2017). Seismological Society. *Annual Meeting Announcement, 88*, 3-30.

Brownlee, J. (2020). *Generative Adversarial Networks with Python.*

Commer, M. (2020). Geophysical Journal International (GEOPHYS J INT). *Climate Research*, 80-100.

Cote, M.-P., Hartman, B., Mercier, O., Meyers, J., Cummings, J., & Harmon, E. (2020). Synthesizing Property & Causalty Ratemaking Datasets using Generative Adversarial Networks. 1-20.

Creswell, A., White, T., Dumoulin, V., Arulkumaran, K., Sengupta, B., & Bharath, A. A. (2018). Generative Adversarial Networks: An Overview.

Dharaneeshwaran, N., Nithya , S., Srinivasan, A., & Senthilkumar, M. (2017). Calculating the User-item Similarity using Pearson's and Cosine Correlation. *International Conference on Trends in Electronics and Informatics*.

Dror, D. M. (2020). *Health Microinsurance - Implementing Universal Health Coverage in the Informal Sector.* World Scientific Publishing Company.

Estevao, J. M. (2019, November 25). An Integrated Computational Approach for Seismic Risk Assessment of Individual Buildings. *Evaluation and Mitigation of Seismic Risk for Existing Buildings, Structures, and Infrastructures, 9*(23), 1-24. doi:doi.org/10.3390/app9235088

Field, E. H. (2005). Probabilistic seismic hazard analysis (PSHA): A primer. 2011.

Global Earthquake Model. (2020). *OpenQuake*. Retrieved from https://www.globalquakemodel.org/openquake

GNS Science. (n.d.). *GNS Science - Te Pu Ao*. Retrieved from https://www.gns.cri.nz/

Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., . . . Bengio, Y. (2014). *Generative Adversarial Networks.* University of Montreal.

Han, J., Kamber, M., & Pei, J. (2012). *Data Mining (Third Edition).* Morgan Kaufmanm. doi:doi.org/10.1016/B978-0-12-381479-1.00002-2

Hui, J. (2018, 01 20). *GAN — What is Generative Adversarial Networks GAN?* Retrieved from www.jonathan-hui.medium.com/: https://jonathan-hui.medium.com/gan-whats-generative-adversarial-networks-and-its-application-f39ed278ef09

Karlsson, A., & Sjöberg, T. (2020). Synthesis of Tabular Financial Data using Generative Adversarial Networks.

Kuo, K. (2019). Generative Synthesis of Insurance Datasets.

Li, Y., Ku, B., Zhang, S., Ahn, J.-K., & Ko, H. (2020). Seismic Data Augmentation Based on Conditional Generative Adversarial Networks. *Sensors*, 1-13.

MIT Data To AI Lab. (n.d.). *SDV - The Synthetic Data Vault*. Retrieved from SDV: https://sdv.dev/SDV/index.html#

Mittelhammer, R. C. (2013). *Mathematical Statistics for Economics and Business.* United States: Springer.

New Zealand Parliament. (2018). *Economic effects of the Canterbury earthquakes* . Retrieved from New Zealand Parliament: https://www.parliament.nz/en/pb/research-papers/document/00PlibCIP051/economic-effects-of-the-canterbury-earthquakes

Nissen-Meyer, T., Sigloch, K., & Schwarz, B. (2017). *Adapting Controlled-source Coherence Analysis to Dense Array Data in Earthquake Seismology.* United State: American Geophysical Union.

Pagani, M., Garcia, J., Gee, R., Johnson, K., Poggi, V., Styron, R., . . . Weatherill, G. (2018). Progress in the compilation of GEM's global mosaic of hazard models. *Earthquake Engineering Thessaloniki, 16*, 1-11.

Power, W., Wilson, K., Bell, R., Robinson, R., Townend, J., Litchfield, N., . . . Wallace, L. M. (2010). Characterizing the seismogenic zone of a major plate boundary subduction thrust: Hikirangi margin, New Zealand. *American Geophysical Union*.

Smith, N. (2017, January 25). *Better managing New Zealand's earthquake risks.* Retrieved December 07, 2020, from beehive.govt.nz: www.beehive.govt.nz/speech/better-managing-new-zealand%E2%80%99s-earthquake-risks

Survey, B. G. (2019). Comparing seismic hazard software packages: M3C vs. OpenQuake. *Earth Hazards and Observatories Programme - Open Report*, 2-12.

Sutherland, R., Reyners, M., Henrys, S., Barker, D. H., Bannister, S., Eberhart-Phillips, D., & Williams, C. A. (2013). Revised Interface Geometry for the Hikurangi Subduction Zone, New Zealand. *Seismological Research Letters, 6*, 1066-1073. doi:doi.org/10.1785/0220130035

U. S. Department of Agriculture. (2019). *Soil Survey Manual (U.S. Department of Agriculture Handbook No. 18).* United States: Lulu.

University of Canterbury. (2020). *Data Management Policy.* Christchurch, New Zealand: UC Policy Library.

University of Canterbury. (2020). *Research Conduct Policy.* Christchurch, New Zealand: UC Policy Library.

University of Canterbury. (2021, 01 25). *Storage*. Retrieved from UC IT Services: https://www.canterbury.ac.nz/its/knowledge-base/storage/

Wallace, L. M., Reyners, M., Cochran, U., Bannister, S., Barnes, M. P., Berryman, K., . . . Sutherlan. (2009). Characterizing the seismogenic zone of a major plate boundary subduction thrust: Hikurangi Margin, New Zealand. *Geochemistry, Geophysics, Geosystems*.

Wang, T., Zhang, Z., & Li, Y. (2019). EarthquakeGen: Earthquake generator using generative adversarial networks. *SEG Technical Program Expanded Abstracts*. doi:doi.org/10.1190/segam2019-3216687.1

Williams, C. A., Eberhart-Phillips, D., Bannister, S., Barker, D. H., Henrys, S., Reyners, M., & Rupert, S. (2013). Revised Interface Geometry for the Hikurangi Subduction Zone, New Zealand. *Seismological Research Letters*, 1066–1073.

Xu, L., & Veeramachaneni, K. (2018). Synthesizing Tabular Data using Generative Adversarial Networks. 1-9.

Xu, L., Skoularidou, M., Cuesta-Infante, A., & Veeramachaneni, K. (2019). Modeling Tabular Data using Conditional GAN. *Conference on Neural Informaiton Processing Systems 2019*, 1-15.

Zahrotun, L. (2016). Coparison Jaccard similarity, Cosine Similarity and Combined both of the Data Clustering with Shared Near Neighbor Method. *Computer Engineering and Applications, 5*.

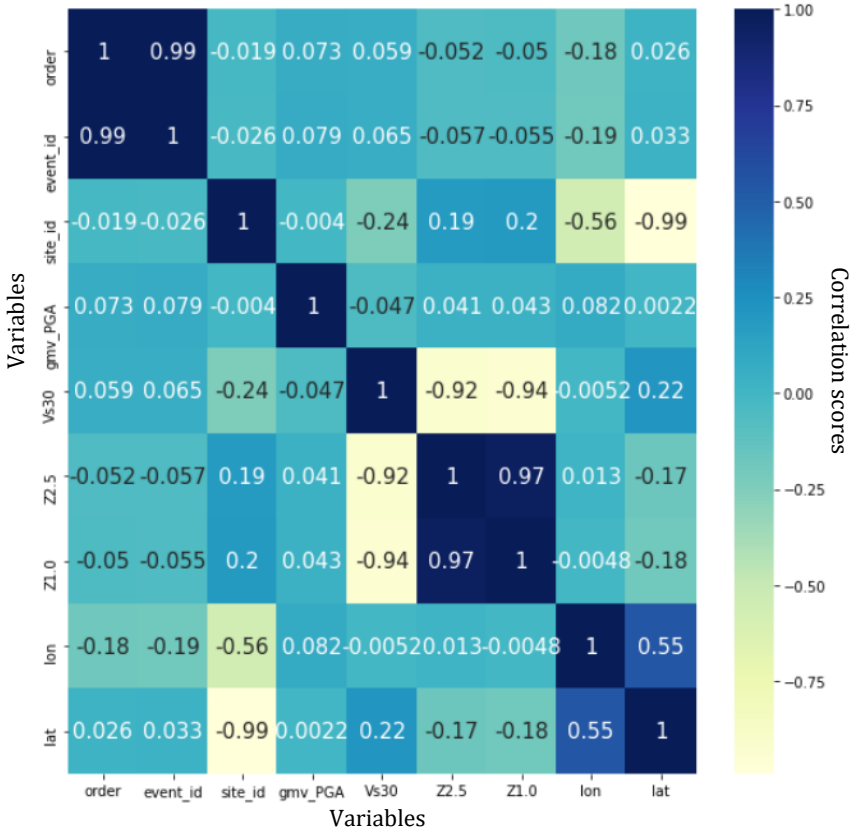
Appendix

Appendix 1. The full dataset

	Unnamed: 0	event_id	site_id	gmv_PGA	Vs30	Z2.5	Z1.0	lon	lat
0	0	0	1244.0	0.010817	530.593	0.915348	90.217180	177.362655	-38.809237
1	1	0	1256.0	0.010859	635.389	0.744794	46.398606	177.482655	-38.809237
2	2	0	1261.0	0.010780	1000.000	0.443316	8.266183	177.532655	-38.809237
3	3	0	1466.0	0.010998	1000.000	0.443316	8.266183	177.592655	-38.829237
4	4	0	1561.0	0.013426	1000.000	0.443316	8.266183	177.472655	-38.839237
...	...	...	...	...	...	...	...	...	...
1927169	1927169	996	9186.0	0.011678	1000.000	0.443316	8.266183	176.192655	-39.459237
1927170	1927170	996	9195.0	0.011773	1000.000	0.443316	8.266183	176.282655	-39.459237
1927171	1927171	996	9199.0	0.014754	1000.000	0.443316	8.266183	176.322655	-39.459237
1927172	1927172	996	9200.0	0.012995	1000.000	0.443316	8.266183	176.332655	-39.459237
1927173	1927173	9	NaN	NaN	NaN	NaN	NaN	NaN	NaN

1927174 rows × 9 columns

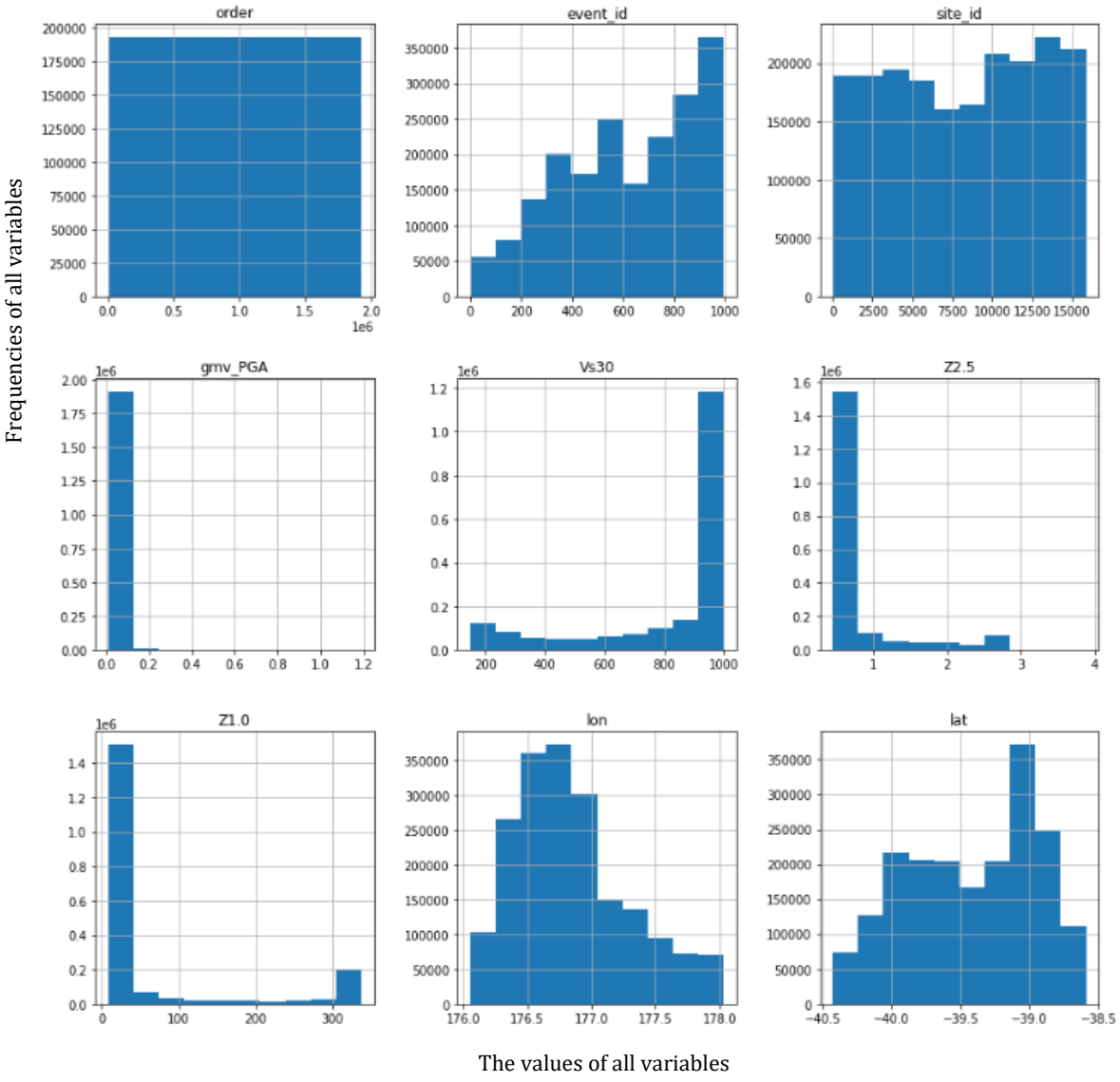
Appendix 2. The correlation of the full dataset



Appendix 3. The variances of the full dataset

```
Variance of event_id:  70395.65112233695
Variance of site_id:  21939811.517852794
Variance of PGA:  0.0004955774281478562
Variance of Vs30:  69194.80803572982
Variance of Z2.5:  0.3905959643205111
Variance of Z1.0:  11291.573466304819
Variance of lon:  0.19675849272752186
Variance of lat:  0.2188909648144187
```

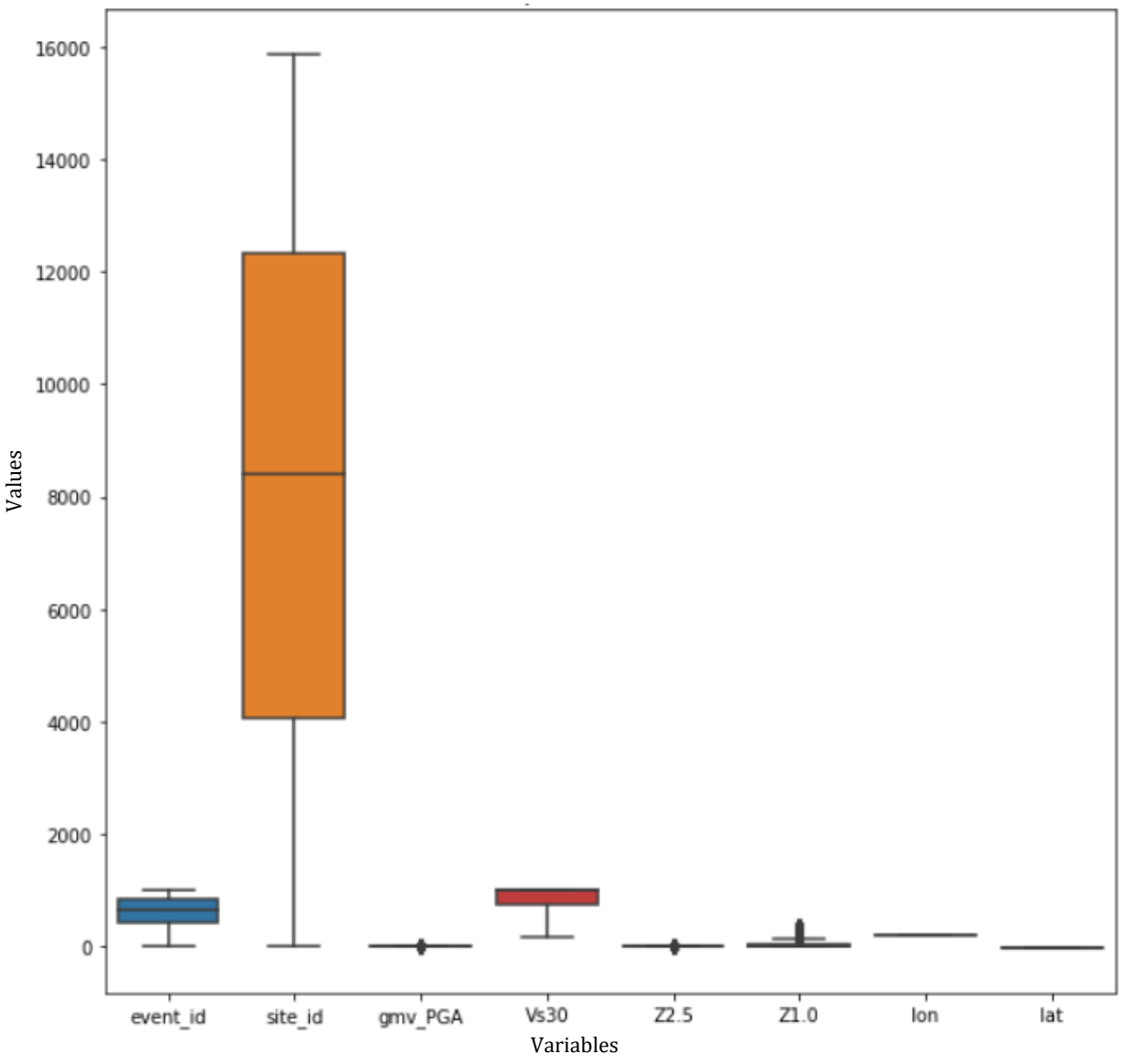
Appendix 4. The distributions of the full dataset



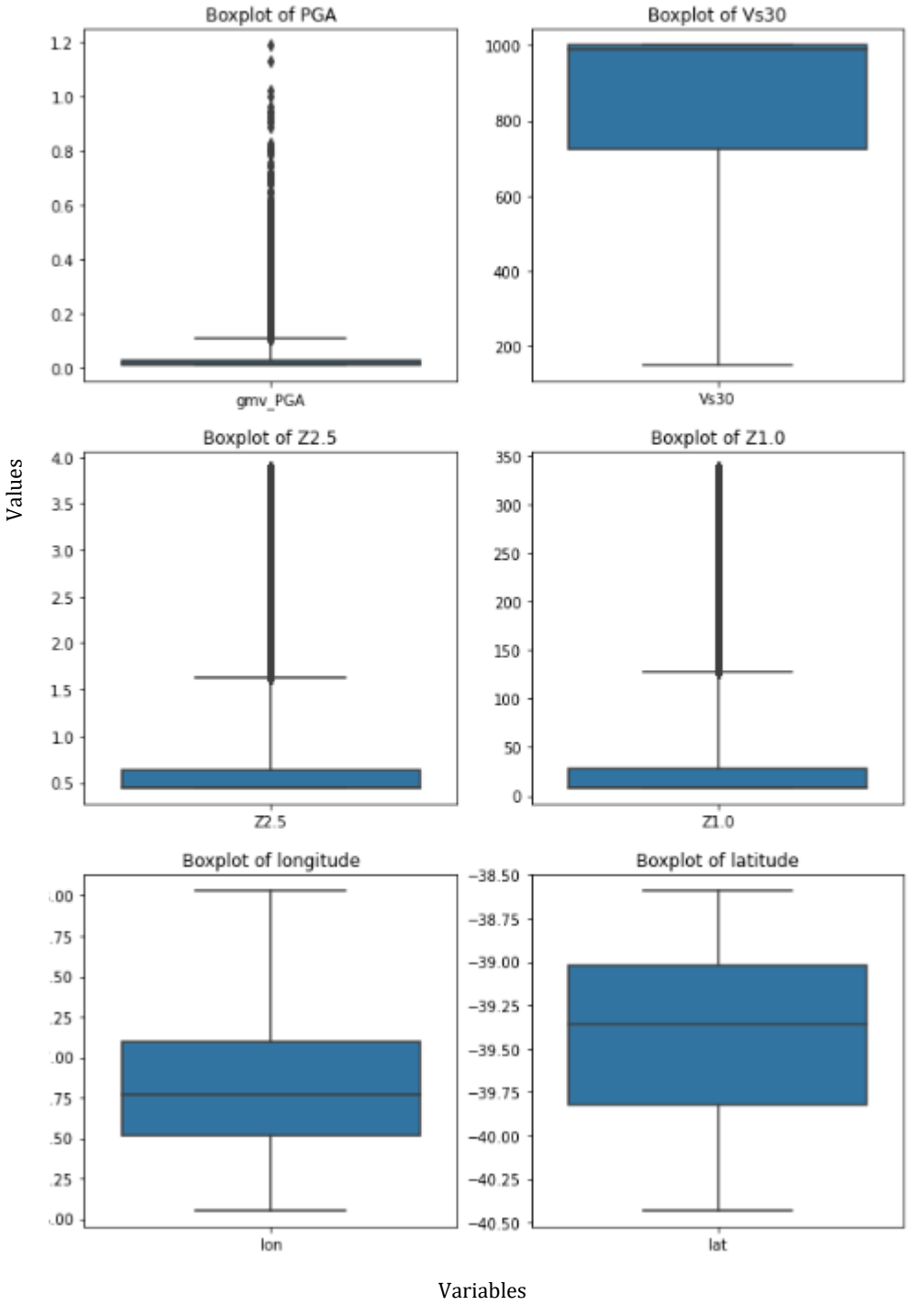
Appendix 5. The statistical summary of the full dataset

	event_id	site_id	gmv_PGA	Vs30	Z2.5	Z1.0	lon	lat
count	1.927173e+06	1.927173e+06	1.927173e+06	1.927173e+06	1.927173e+06	1.927173e+06	1.927173e+06	1.927173e+06
mean	6.201857e+02	8.168213e+03	2.530335e-02	8.239278e+02	7.451369e-01	5.977937e+01	1.768460e+02	-3.941956e+01
std	2.653218e+02	4.683995e+03	2.226157e-02	2.630491e+02	6.249768e-01	1.062618e+02	4.435747e-01	4.678578e-01
min	0.000000e+00	0.000000e+00	1.000000e-02	1.500000e+02	4.433162e-01	8.266183e+00	1.760527e+02	-4.042924e+01
25%	4.010000e+02	4.050000e+03	1.299682e-02	7.252880e+02	4.433162e-01	8.266183e+00	1.765127e+02	-3.981924e+01
50%	6.260000e+02	8.394000e+03	1.804735e-02	9.883010e+02	4.493247e-01	8.646085e+00	1.767727e+02	-3.935924e+01
75%	8.510000e+02	1.234400e+04	2.882650e-02	1.000000e+03	6.401615e-01	2.812550e+01	1.770927e+02	-3.901924e+01
max	9.960000e+02	1.587700e+04	1.188596e+00	1.000000e+03	3.883874e+00	3.374264e+02	1.780327e+02	-3.858924e+01

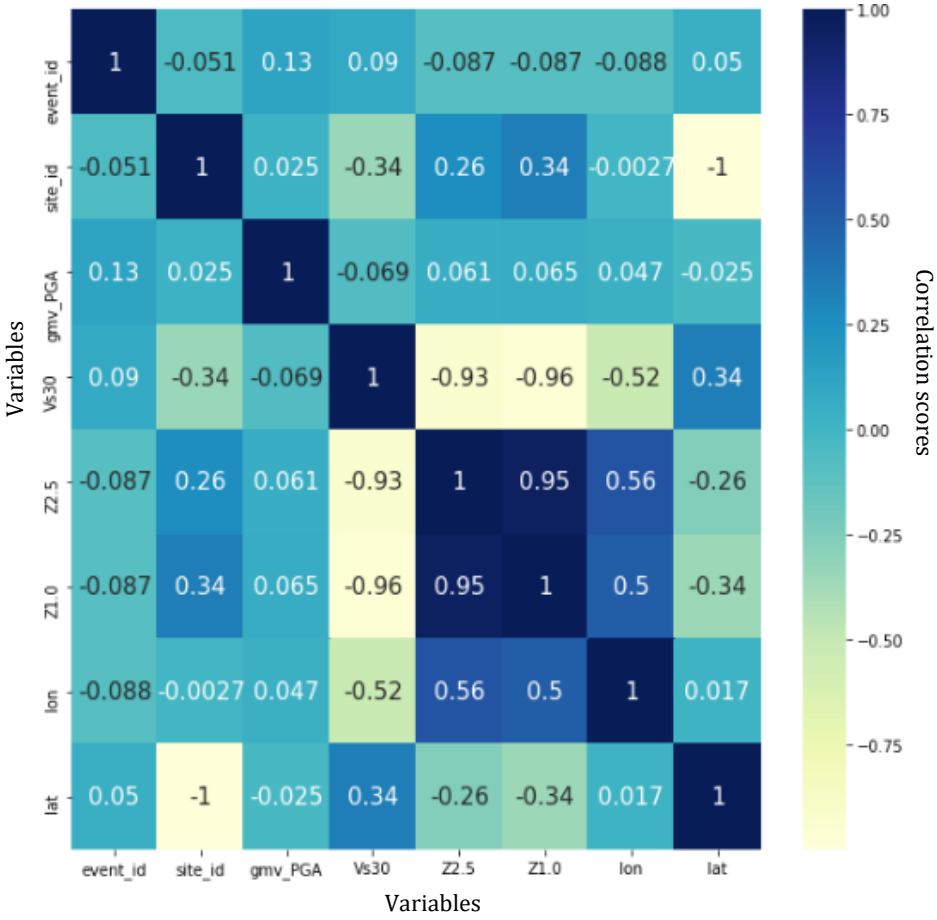
Appendix 6. The boxplots of the full dataset at whisker of 5



Appendix 7. The boxplots of PGA, Vs30, Z2.5, and Z1.0 in the full dataset



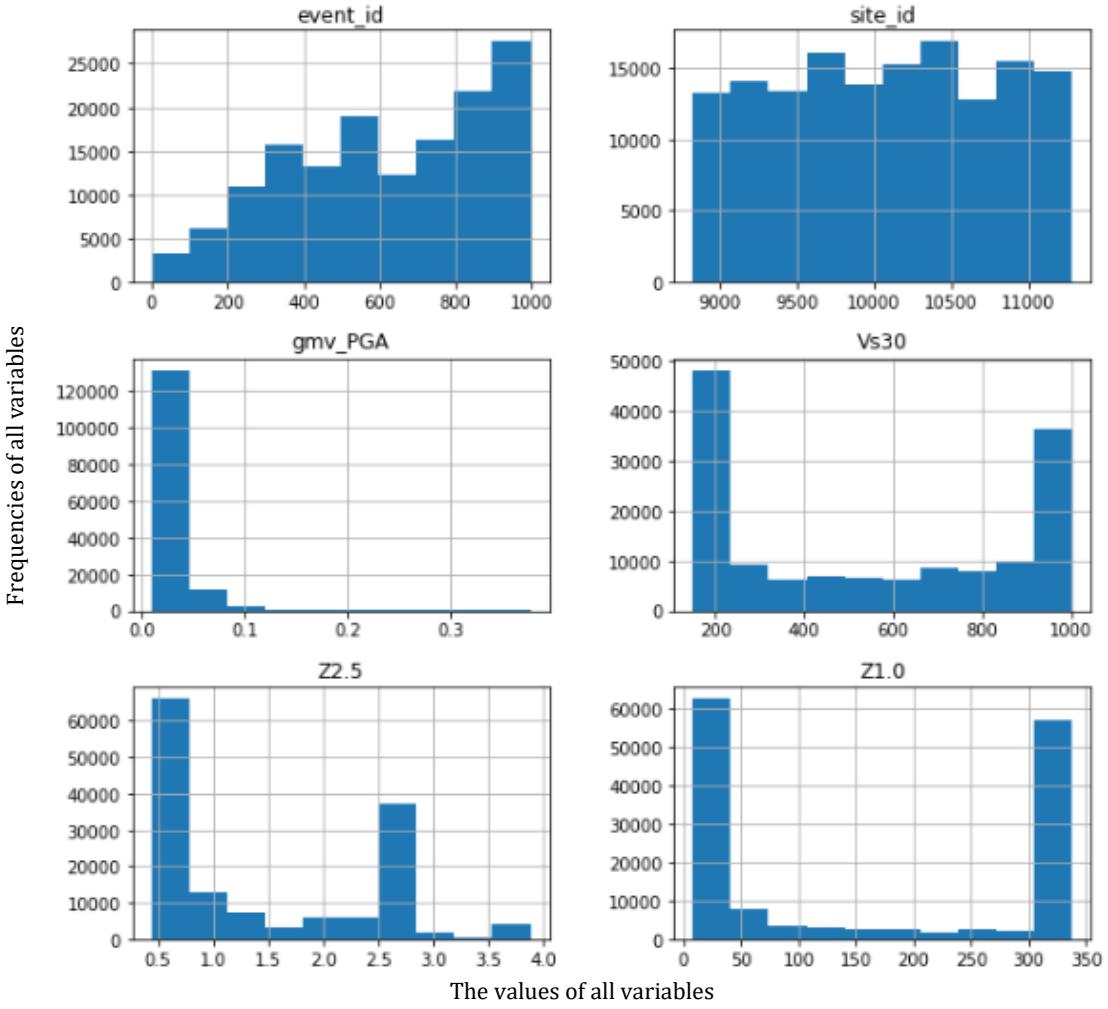
Appendix 8. The correlation of the sub-dataset



Appendix 9. The variances of the sub-dataset

```
event_id      69124.935695
site_id      494605.364514
gmv_PGA           0.000396
Vs30         105073.882663
Z2.5              1.039278
Z1.0          22313.664402
dtype: float64
```

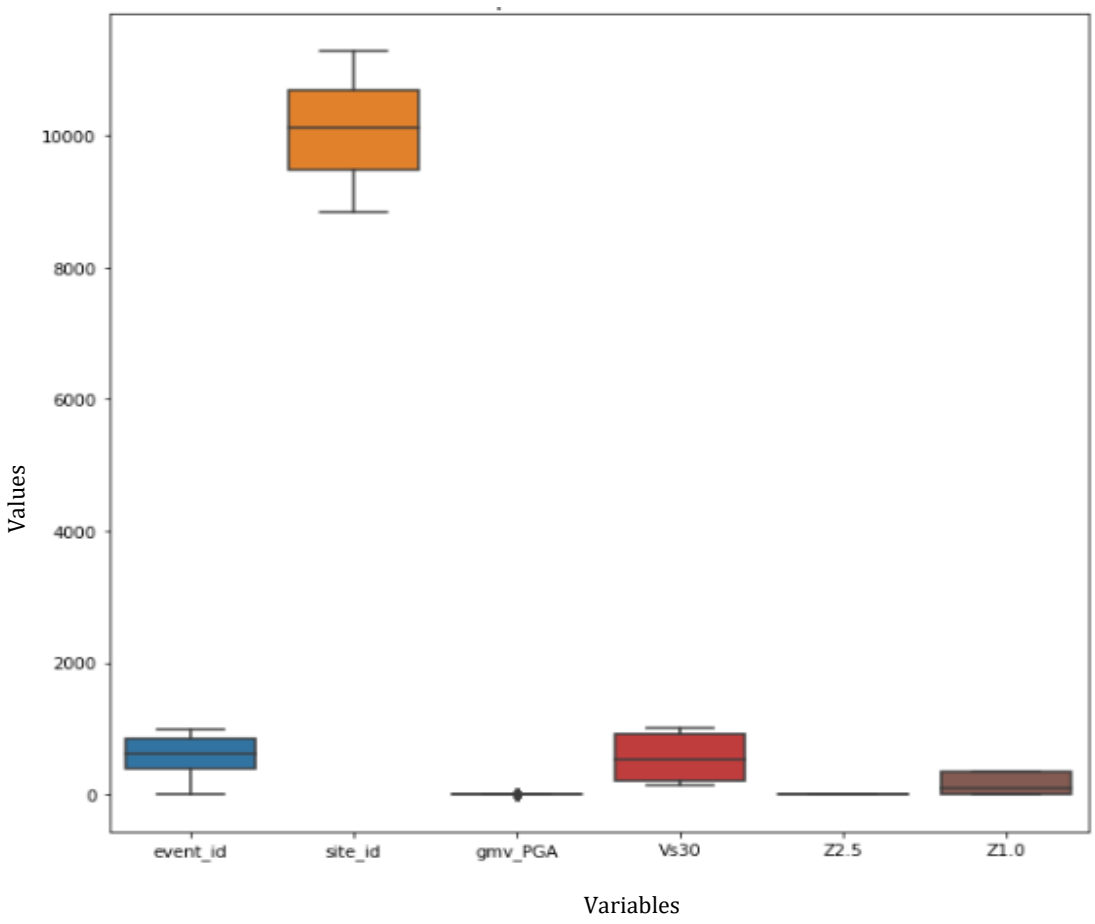
Appendix 10. The distributions of the sub-dataset



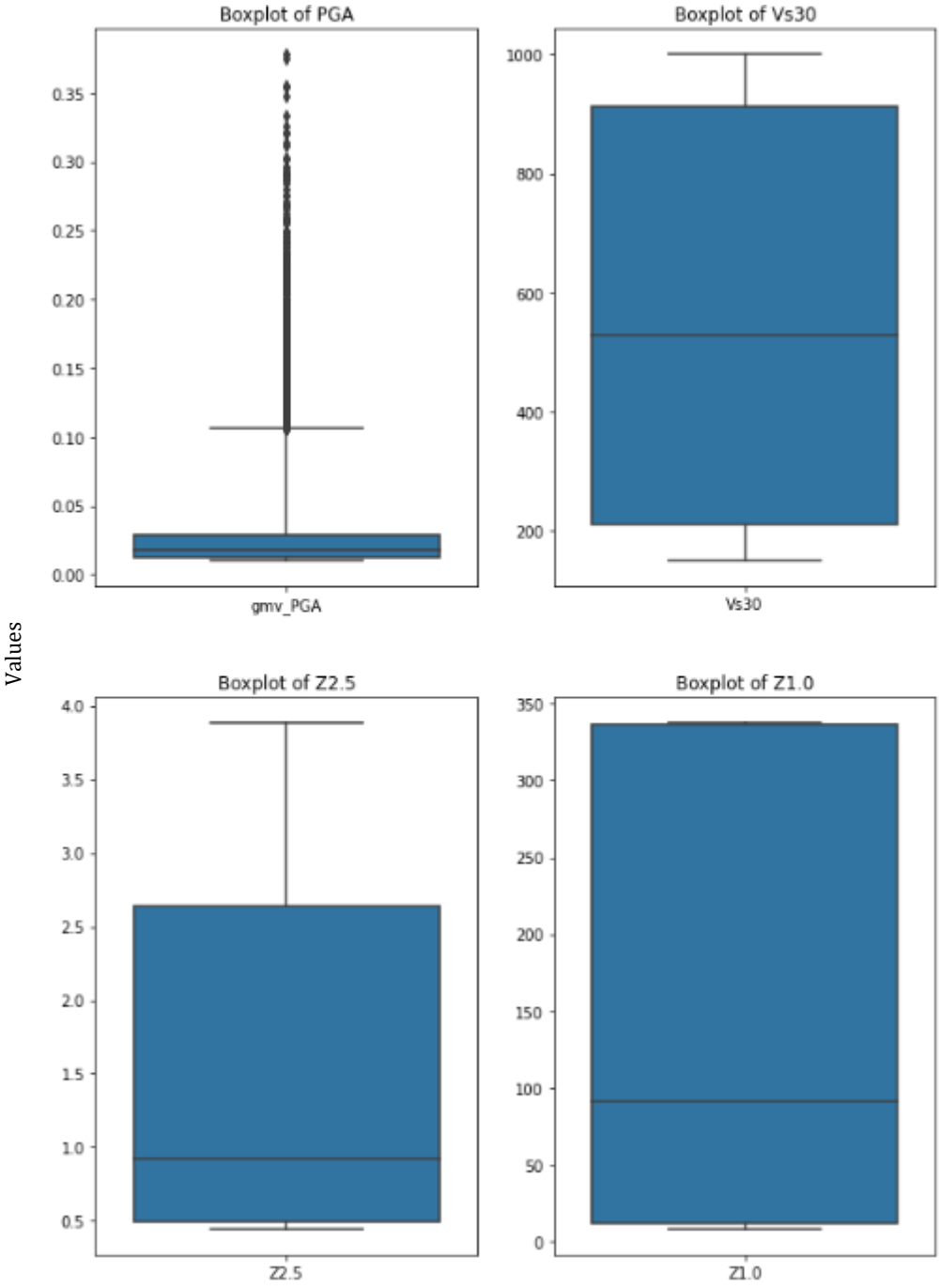
Appendix 11. The statistical summary of the sub-dataset

	event_id	site_id	gmv_PGA	Vs30	Z2.5	Z1.0
count	145756.000000	145756.000000	145756.000000	145756.000000	145756.000000	145756.000000
mean	621.276318	10076.617731	0.024710	559.461941	1.438027	162.575267
std	262.916214	703.281853	0.019898	324.151018	1.019450	149.377590
min	0.000000	8827.000000	0.010000	150.000000	0.443316	8.266183
25%	402.000000	9484.000000	0.012934	210.000000	0.492182	11.717860
50%	626.000000	10106.000000	0.017909	528.542000	0.919413	91.475169
75%	851.000000	10693.000000	0.028498	912.650000	2.642985	336.092436
max	996.000000	11280.000000	0.377790	1000.000000	3.883874	337.426407

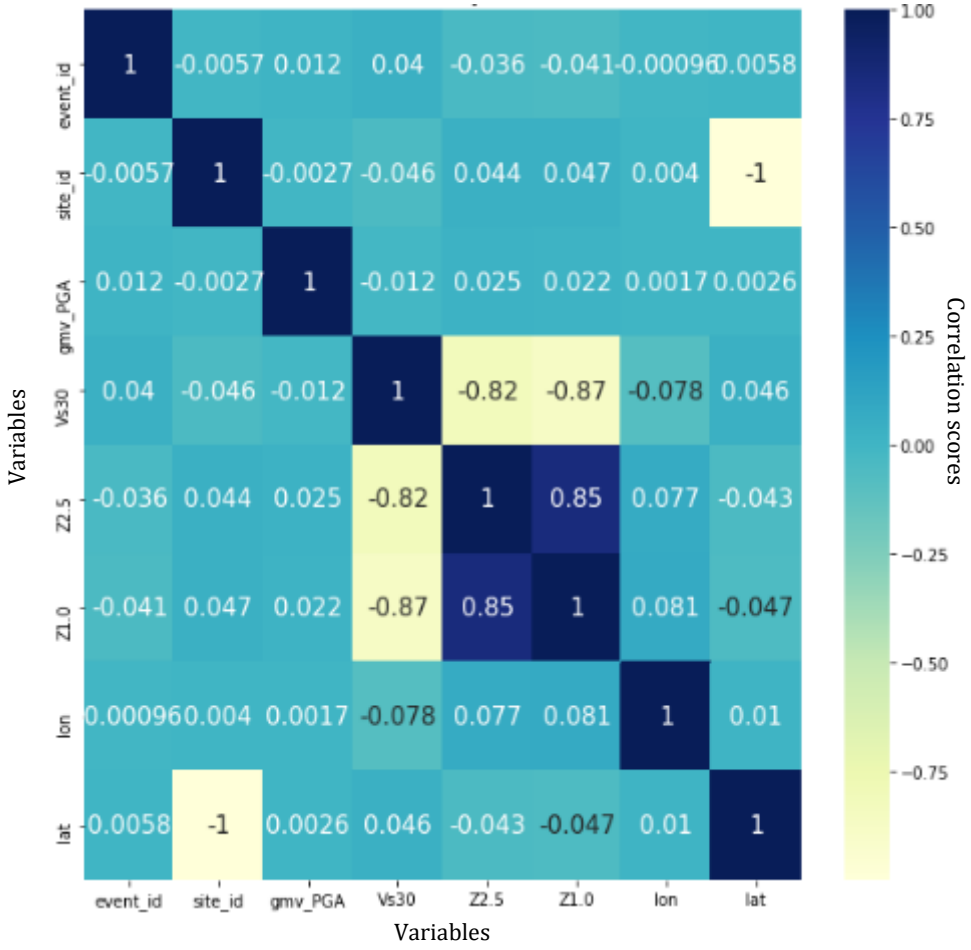
Appendix 12. The boxplots of the sub-dataset



Appendix 13. The boxplots of PGA, Vs30, Z2.5, and Z1.0 in the sub-dataset



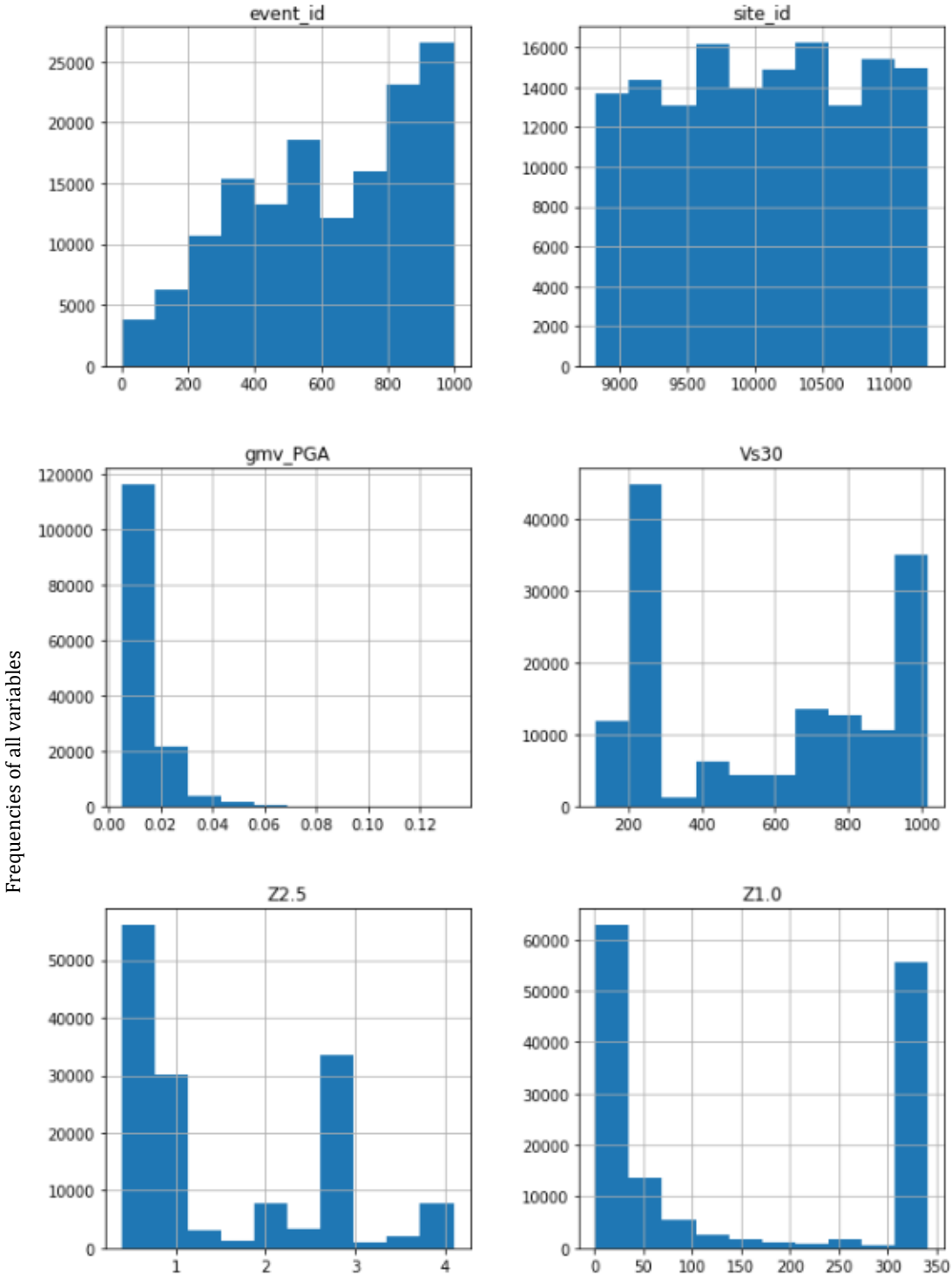
Appendix 14. The correlation of the synthetic dataset



Appendix 15. The variances of the synthetic dataset

event_id	69852.456058
site_id	499954.068459
gmv_PGA	0.000082
Vs30	106419.116658
Z2.5	1.189234
Z1.0	22345.812858

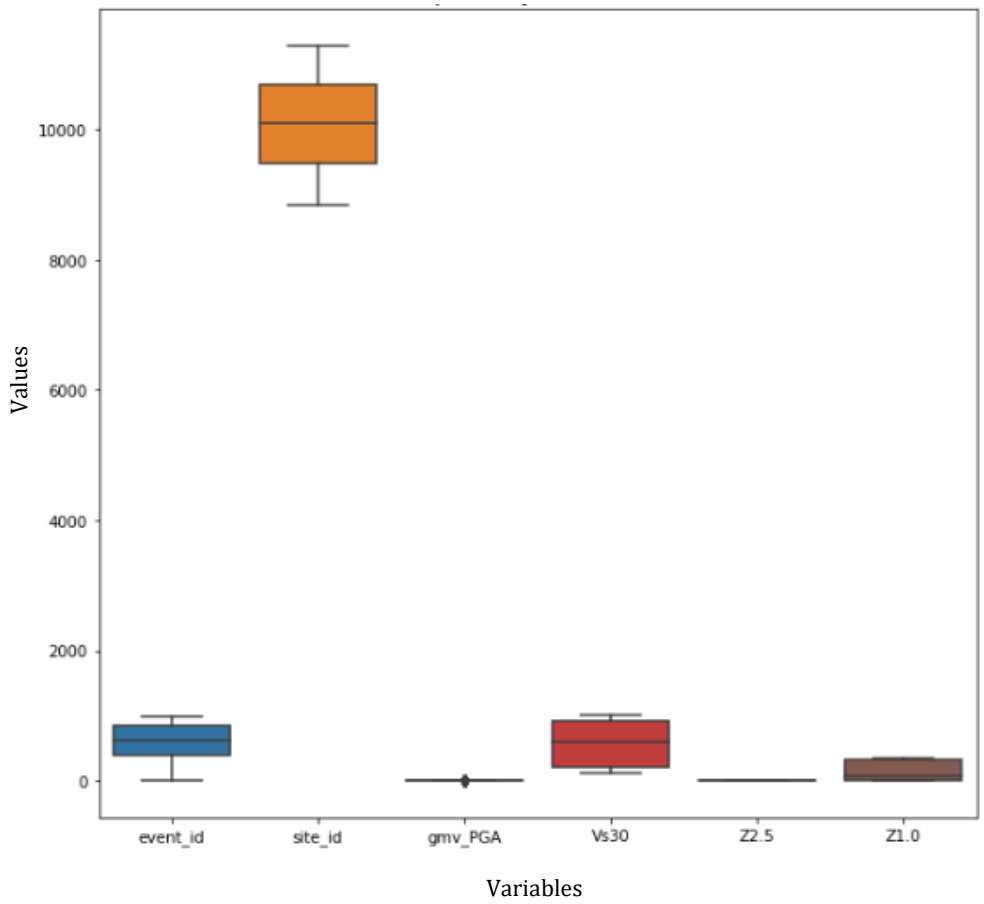
Appendix 16. The distributions of the synthetic dataset



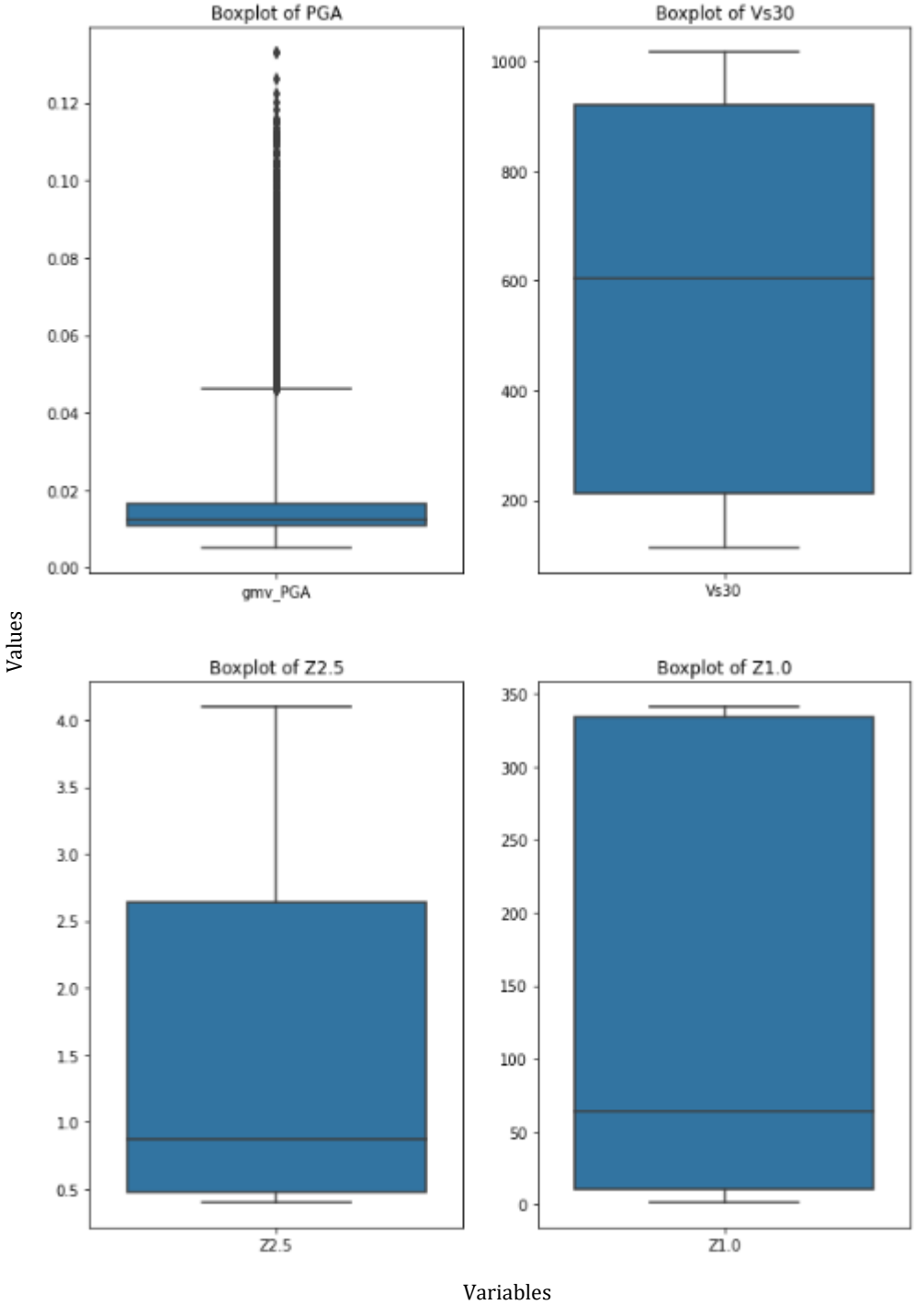
Appendix 17. The statistical summary of the synthetic dataset

	event_id	site_id	gmv_PGA	Vs30	Z2.5	Z1.0
count	145756.000000	145756.000000	145756.000000	145756.000000	145756.000000	145756.000000
mean	619.155939	10073.050571	0.015306	575.922746	1.466756	151.614623
std	264.296152	707.074302	0.009058	326.219430	1.090520	149.485159
min	2.000000	8827.000000	0.005014	112.642875	0.399223	1.421507
25%	402.000000	9479.000000	0.010616	212.323050	0.473228	10.005165
50%	626.000000	10100.000000	0.012219	603.649620	0.876071	64.061889
75%	851.000000	10693.000000	0.016560	918.652104	2.636870	333.868330
max	996.000000	11280.000000	0.133082	1015.790346	4.099280	341.122643

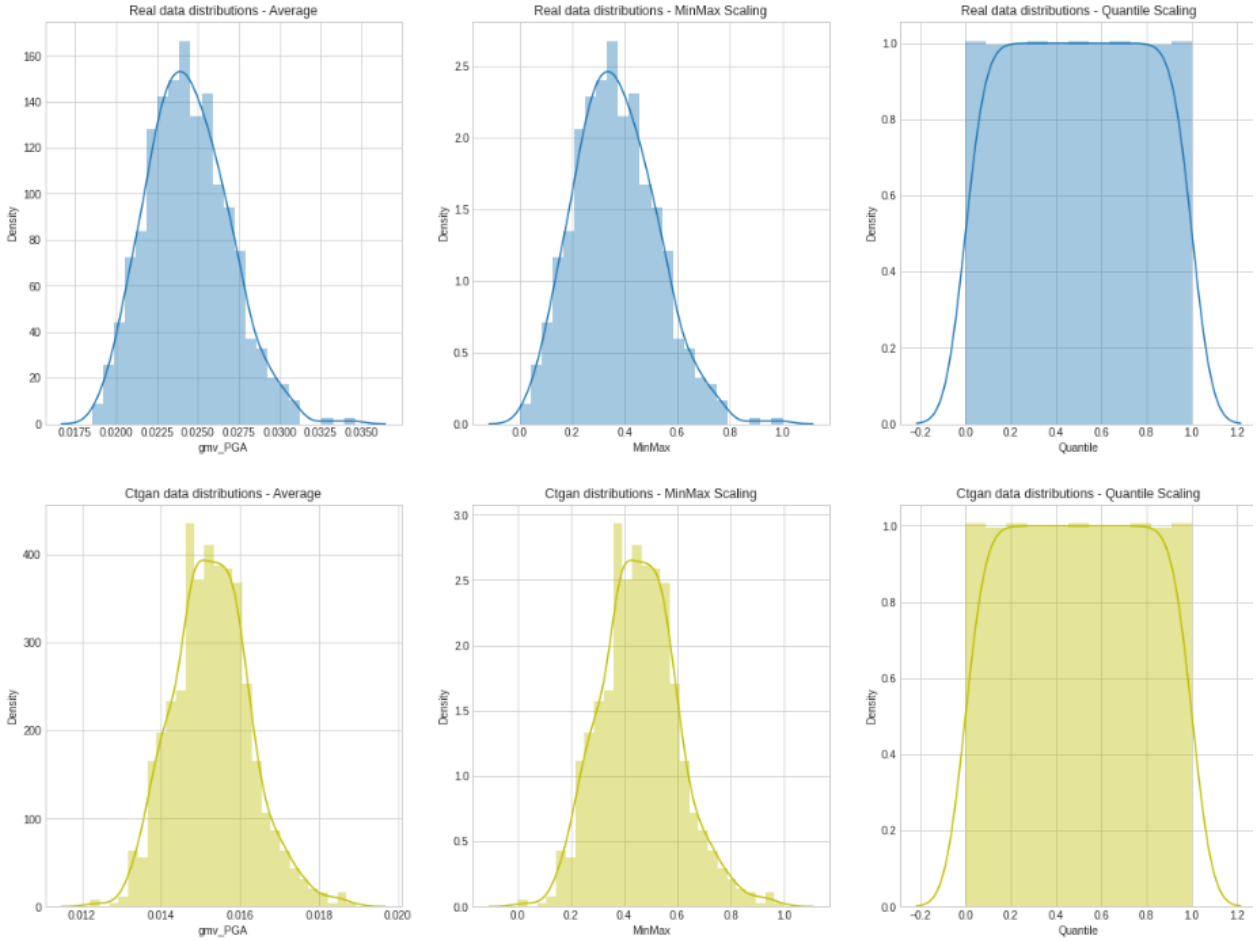
Appendix 18. The boxplots of the synthetic dataset



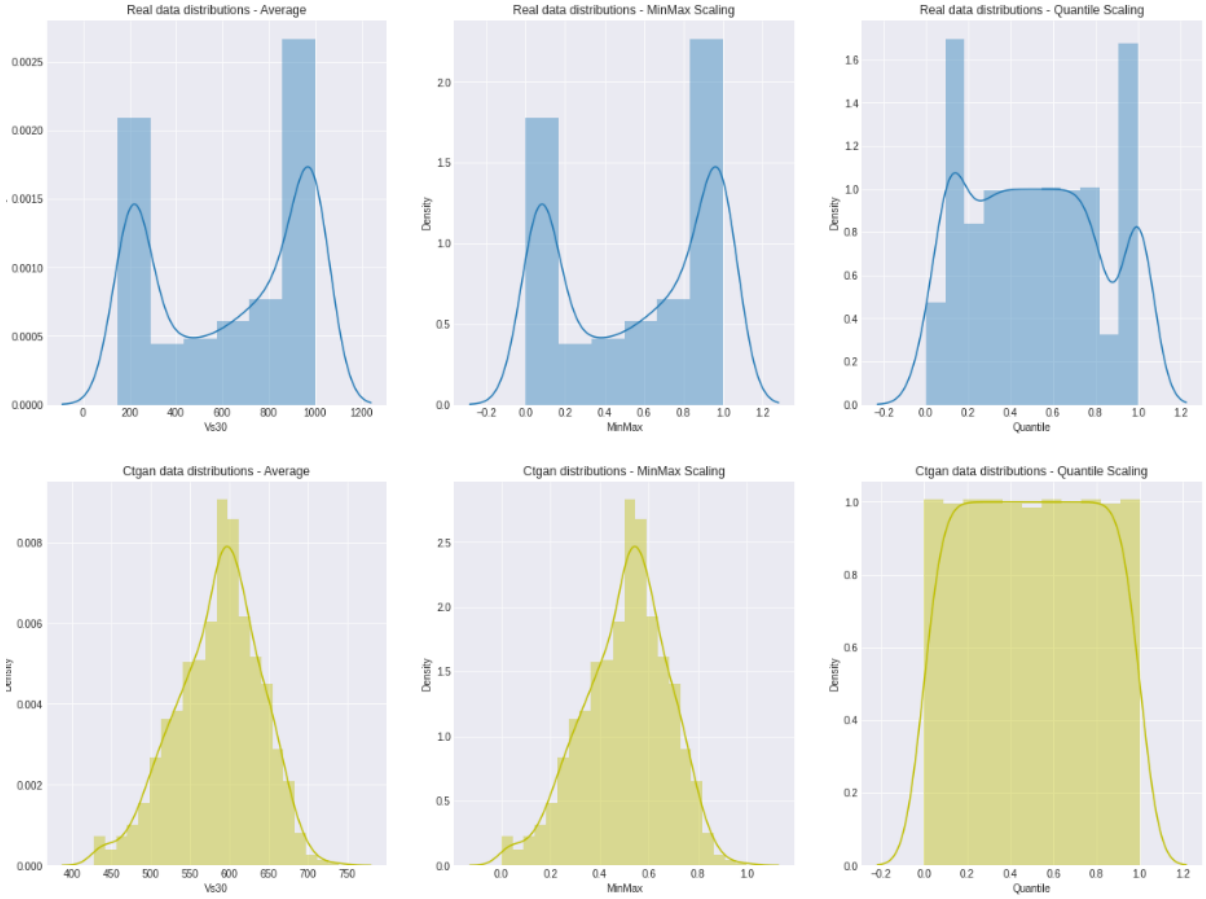
Appendix 19. The boxplots of PGA, Vs30, Z2.5, Z1.0 in the synthetic dataset



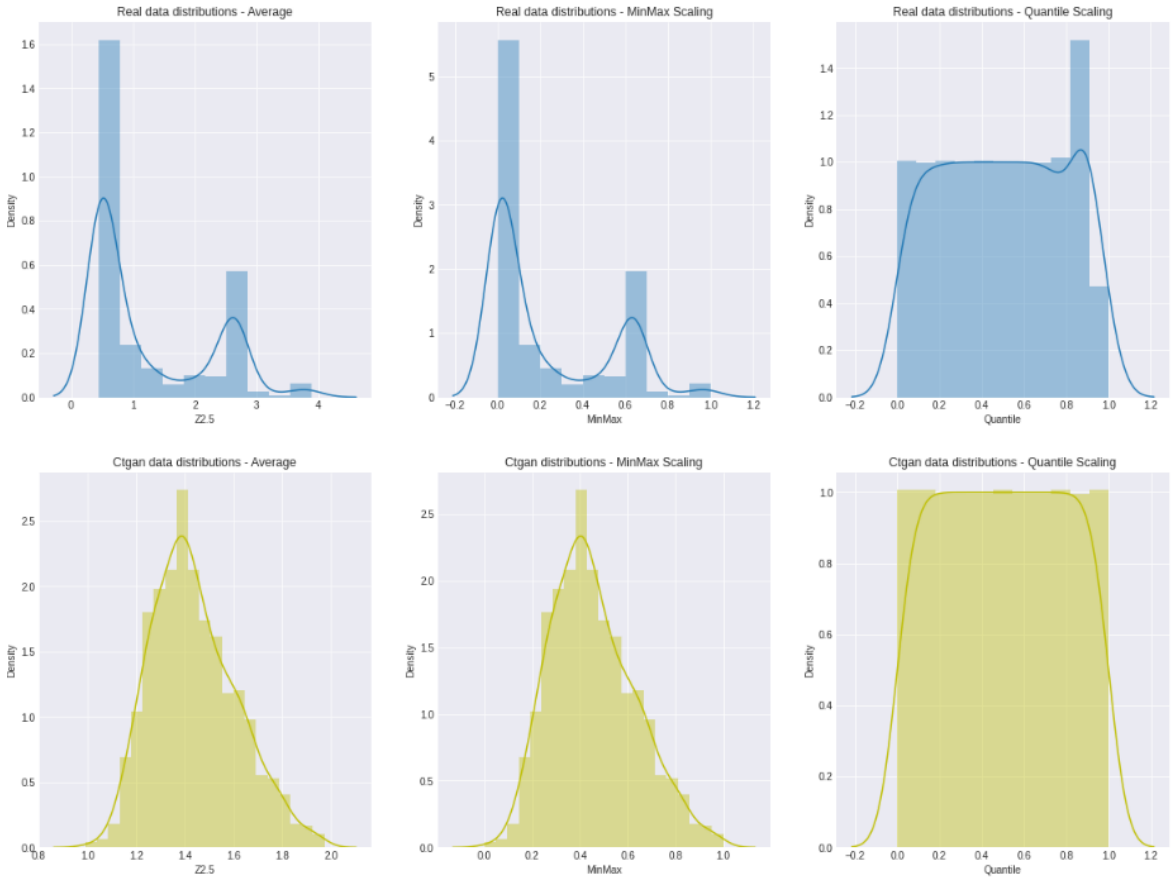
Appendix 20. The distributions when averaging and scaling of PGA (above - sub-dataset, below- synthetic dataset)



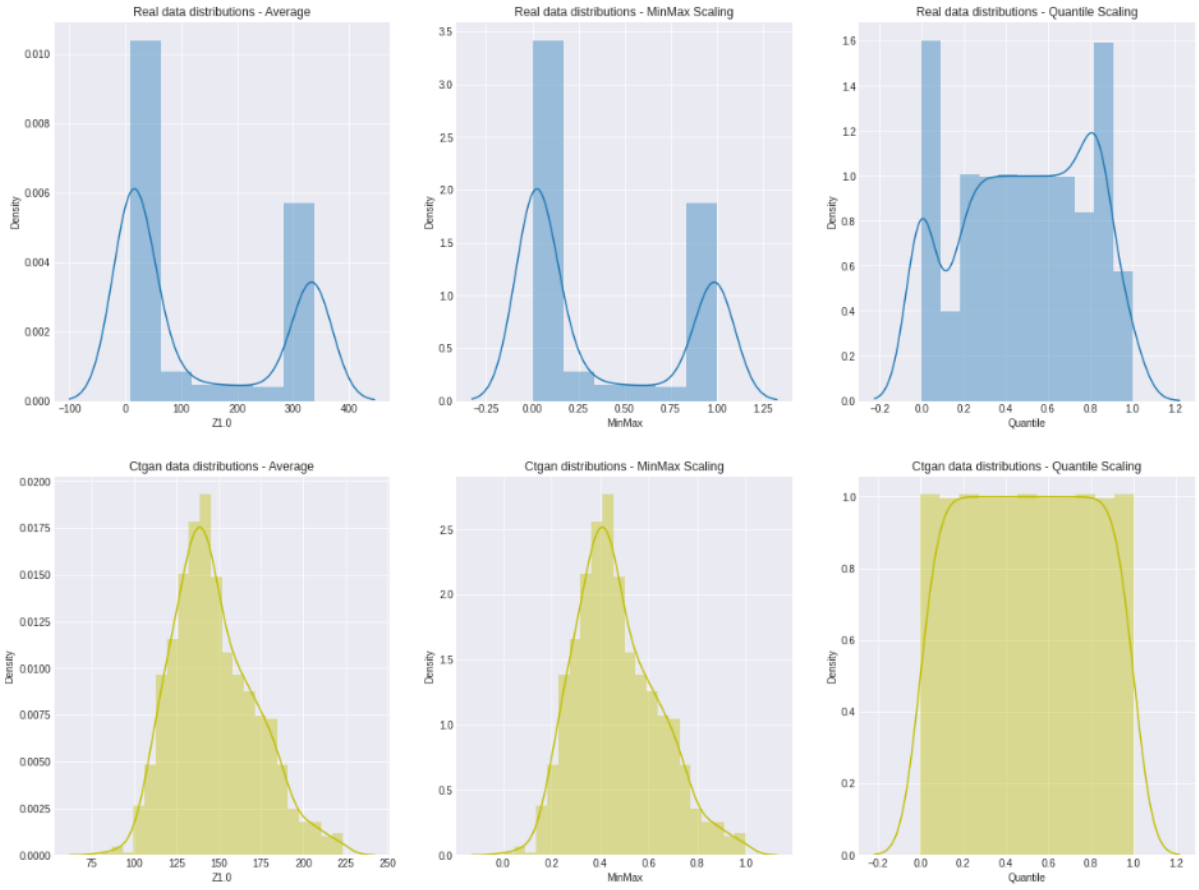
Appendix 21. The distributions when averaging and scaling of Vs30 (above - sub-dataset, below- synthetic dataset)



Appendix 22. The distributions when averaging and scaling of Z2.5 (above - sub-dataset, below- synthetic dataset)

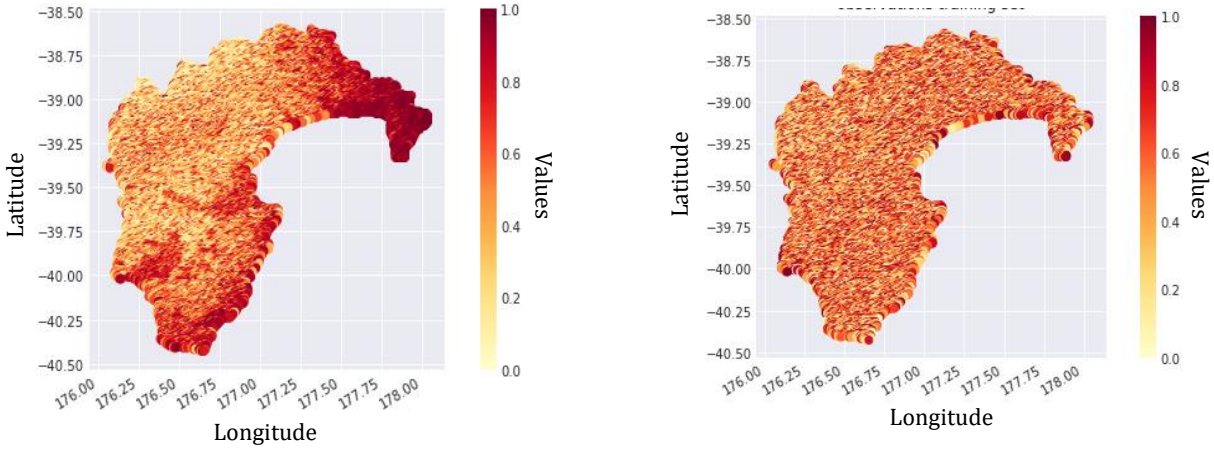


Appendix 23. The distributions when averaging and scaling of Z1.0 (above - sub-dataset, below- synthetic dataset)

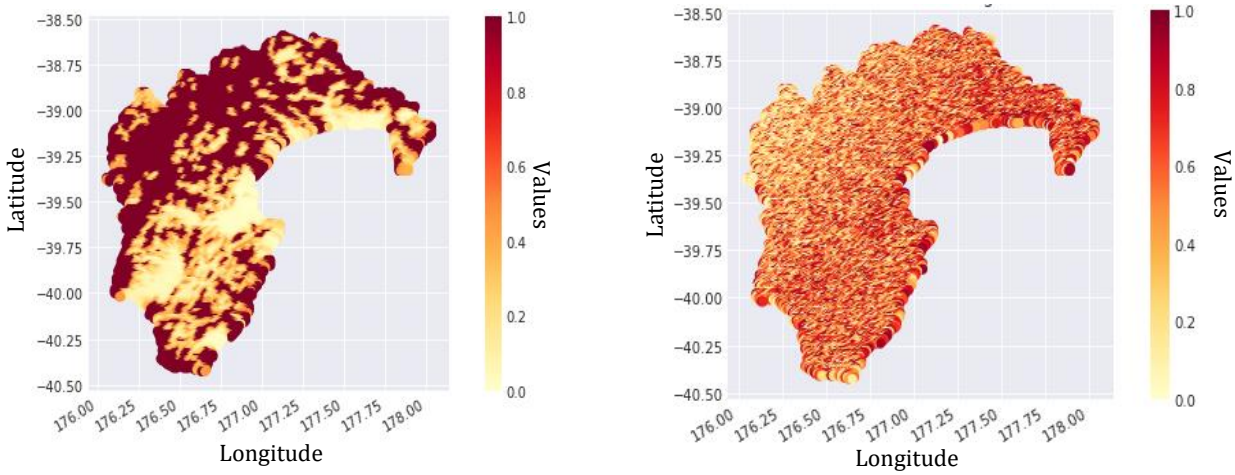


Appendix 24. The Correlation scores (when trying the script on the full dataset)

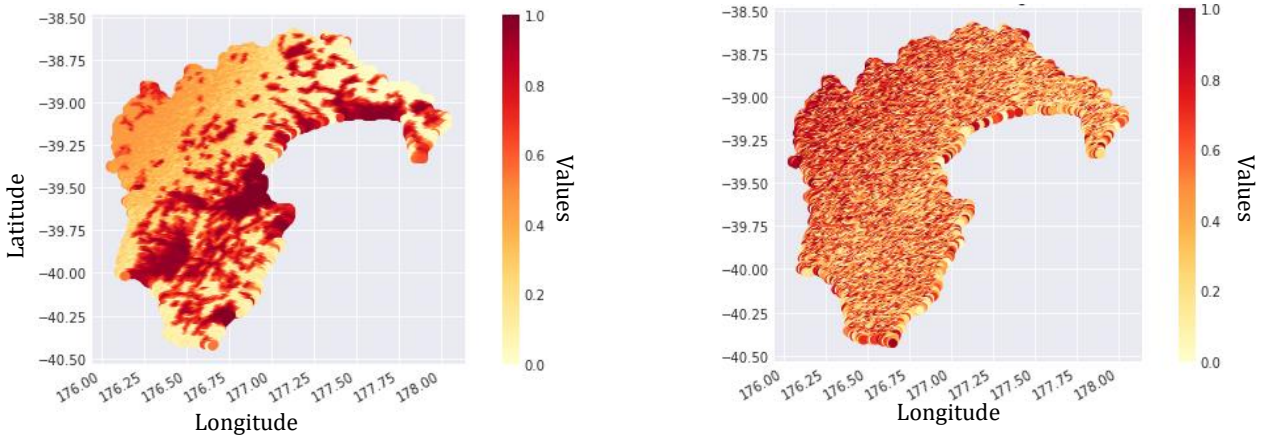
Variable names	Pearson's Correlation	Spearman's Correlation
Event_id	0.820465	0.889159
Site_id	0.174601	0.324694
Gmv_PGA	0.044864	0.051138
Vs30	-0.172997	-0.168036
Z2.5	-0.138978	-0.083244
Z1.0	-0.151406	-0.171003



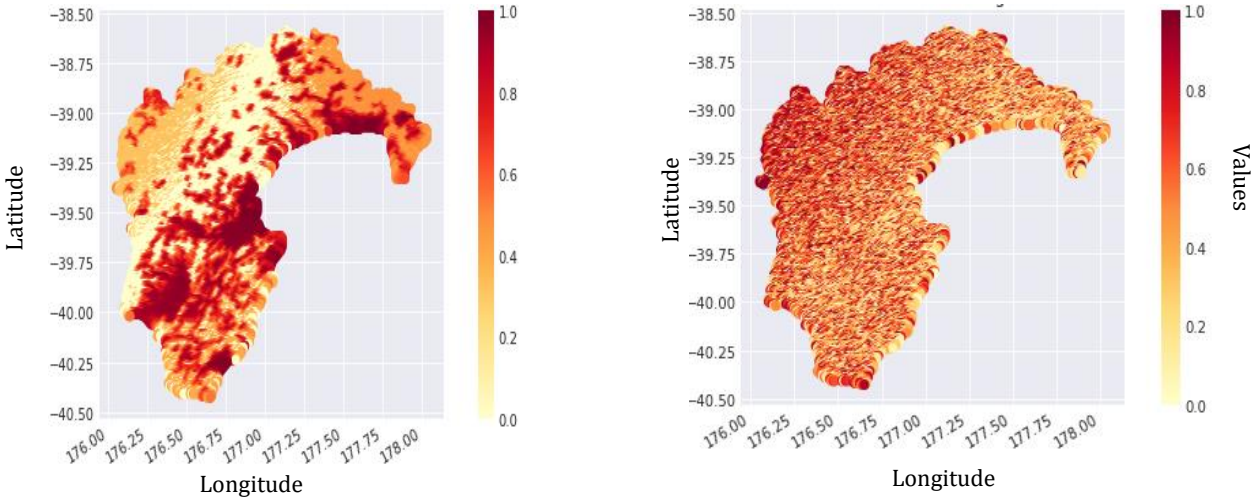
Appendix 25. The maps of PGA in the full dataset (left - full dataset, right - synthetic dataset)



Appendix 26. The maps of Vs30 in the full dataset (left - full dataset, right - synthetic dataset)



Appendix 27. The maps of Z2.5 in the full dataset (left - full dataset, right - synthetic dataset)



Appendix 28. The maps of Z1.0 in the full dataset (left - full dataset, right - synthetic dataset)

Appendix 29. Data ethics (section explanation)

Criteria		Explanation
A. Data Collection	A.1. Informed consent	If there are human subjects, have they given informed consent, where subjects affirmatively opt-in and have a clear understanding of the data uses to which they consent?
	A.2. Collection bias	Have we considered sources of bias that could be introduced during data collection and survey design and taken steps to mitigate those?
	A.3. Limit PII exposure	Have we considered ways to minimize exposure of personally identifiable information (PII) for example through anonymization or not collecting information that isn't relevant for analysis?
	A.4. Downstream bias mitigation	Have we considered ways to enable testing downstream results for biased outcomes (e.g., collecting data on protected group status like race or gender)?
B. Data Storage	B.1. Data security	Do we have a plan to protect and secure data (e.g., encryption at rest and in transit, access controls on internal users and third parties, access logs, and up-to-date software)?
	B.2. Right to be forgotten	Do we have a mechanism through which an individual can request their personal information be removed?
	B.3. Data retention plan	Is there a schedule or plan to delete the data after it is no longer needed?
C. Analysis	C.1. Missing perspectives	Have we sought to address blindspots in the analysis through engagement with relevant stakeholders (e.g., checking assumptions and discussing implications with affected communities and subject matter experts)?
	C.2. Dataset bias	Have we examined the data for possible sources of bias and taken steps to mitigate or address these

		biases (e.g., stereotype perpetuation, confirmation bias, imbalanced classes, or omitted confounding variables)?
	C.3. Honest representation	Are our visualizations, summary statistics, and reports designed to honestly represent the underlying data?
	C.4. Privacy in analysis	Have we ensured that data with PII are not used or displayed unless necessary for the analysis?
	C.5. Auditability	Is the process of generating the analysis well documented and reproducible if we discover issues in the future?
D. Modeling	D.1. Proxy discrimination	Have we ensured that the model does not rely on variables or proxies for variables that are unfairly discriminatory?
	D.2. Fairness across groups	Have we tested model results for fairness with respect to different affected groups (e.g., tested for disparate error rates)?
	D.3. Metric selection	Have we considered the effects of optimizing for our defined metrics and considered additional metrics?
	D.4. Explainability	Can we explain in understandable terms a decision the model made in cases where a justification is needed?
	D.5. Communicate bias	Have we communicated the shortcomings, limitations, and biases of the model to relevant stakeholders in ways that can be generally understood?
E. Deployment	E.1. Redress	Have we discussed with our organization a plan for a response if users are harmed by the results (e.g., how does the data science team evaluate these cases and update analysis and models to prevent future harm)?

	E.2. Roll back	Is there a way to turn off or roll back the model in production if necessary?
	E.3. Concept drift	Do we test and monitor for concept drift to ensure the model remains fair over time?
	E.4. Unintended use	Have we taken steps to identify and prevent unintended uses and abuse of the model and do we have a plan to monitor these once the model is deployed?